常用库的使用总结

(开始时间：2021/6/13)

(by 孟智超)

1 一、 `OS` **模块**
1.1 os模块简介
1.2 os模块中常用的属性和函数
1.2.1 os.getcwd()
1.2.2 os.chdir()
1.2.3 os.curdir、os.pardir
1.2.4 os.makedirs()
1.2.5 os.removedirs()
1.2.6 os.mkdir()
1.2.7 os.rmdir()
1.2.8 os.listdir()
1.2.9 os.remove()
1.2.10 os.stat()
1.2.11 **os.sep**
1.2.12 **os.pathsep**
1.2.13 **os.system()**
1.2.14 os.environ
1.2.15 os.rename()
1.3 os.path相关
1.3.1 os.path.split(path)
1.3.2 os.path.abspath(path)
1.3.3 os.path.dirname(path)
1.3.4 os.path.basename(path)
1.3.5 os.path.exists(path)
1.3.6 os.path.isabs(path)
1.3.7 os.path.isfile(path)
1.3.8 os.path.isdir(path)
1.3.9 os.path.join(path1， [path2],[path3])
1.3.10 os.path.getatime(path)
1.3.11 os.path.getmtime(time)
1.3.12 os.popen('ls')
1.3.13 os.path.splitext()
1.4 遍历目录
2 二、 `argparse` **模块**
2.1 argparse模块简介
2.2 argparse例子
2.3 argparse 的三个主要函数
2.3.1 **Creating a parser**：
2.3.2 **Adding arguments**：
2.3.3 **Parsing arguments**：
2.4 ArgumentParser对象(ArgumentParser objects)
2.4.1 对象参数简介
2.4.2 常用对象参数具体用法

2.5 add_argument()方法
2.5.1 函数参数简介
2.5.2 常用函数参数具体用法
2.6 parse_args()方法
2.7 namespace 对象
2.8 argparse 实例（以决策树参数、随机森林参数为例）
2.8.1 直接添加
2.8.2 通过配置文件（以随机森林为例，两个参数做测试）
2.9 封装
3 三、 time 和 datetime 模块
3.1 time模块
3.2 datetime模块
4 四、 sys 模块
5 五、 面向对象 回顾
5.1 对面向对象的理解
5.1.1 面向对象的编程（object oriented programming）
5.1.2 python面向对象的术语
5.1.3 函数编程和面向对象编程的区别
5.2 封装、继承、多态
5.2.1 封装（encapsulation）
5.2.2 继承（Inheritance）
5.2.3 多态（Polymorphism）
5.3 面向对象的各种方法
5.3.1 静态方法(装饰器：@staticmethod)
5.3.2 类方法（装饰器：@classmethod）
5.3.3 属性方法(@property)
5.4 高级面向对象
5.4.1 成员修饰符
5.4.2 特殊成员
5.4.3 类与对象
5.4.4 异常处理
5.4.5 反射、自省
5.4.6 单例模式
6 六、 json 模块
6.1 序列化概念
6.2 json标准库
6.2.1 基本用法
6.2.2 json.dumps()
6.2.3 json.loads()
6.2.4 json.dump()
6.2.5 json.load()
6.3 类的序列化
6.4 无法序列化问题
7 七、 pickle 模块
7.1 pickle模块简介
7.1.1 什么是序列化
7.1.2 pickle特点

7.2 pickle模块常用方法
7.2.1 pickle.dumps(obj)
7.2.2 pickle.loads(bytes_object)
7.2.3 pickle.dump(obj , file)
7.2.4 pickle.load(file)
8 八、 `logging` 模块
8.1 日志
8.1.1 作用
8.1.2 日志级别
8.1.3 日志的流程
8.1.4 日志输出格式：
8.2 logging模块简介
8.2.1 logging模块的使用方式介绍
8.3 logging模块基本使用（basicConfig）
8.3.1 basicConfig()方法
8.3.2 捕获异常
8.3.3 格式字符串
8.3.4 经过配置的日志输出
8.3.5 自定义 Logger
8.3.6 其他说明
8.4 logging日志模块相关类及其常用方法介绍
8.5 封装
9 九、 `tqdm` 模块
9.1 tqdm简介
9.1.1 地址
9.1.2 简介
9.2 tqdm 使用
9.2.1 简单使用
9.2.2 设置动态变化的description
9.2.3 结合pandas使用
9.2.4 结合jupyter 使用
9.2.5 tqdm.write()
10 十、sqlite3
10.1 简介
10.2 创建数据库
10.3 创建表
10.4 插入数据
10.5 更新表
10.6 获取所有数据
10.7 列出所有表
10.8 创建-删除表
10.9 批量插入
11 十一、pytesseract
12 十二、schedule
12.1 按时间间隔执行定时任务
12.2 装饰器：通过 @repeat() 装饰静态方法
12.3 传递参数

12.4 取消定时任务
12.5 在指定时间执行一次任务
12.6 根据标签检索、取消任务
12.7 运行任务到某时间
13 十三、rich
14 pathlib
14.1 常用属性和函数
14.2 示例
15 十五、 pydantic

1 一、 `OS` 模块

1.1 os模块简介

- OS模块简单的来说它是一个Python的 `系统编程的操作模块` ， 可以处理文件和目录这些我们日常手动需要做的操作。

  在自动化测试中,经常需要查找操作文件,比如查找配置文件(从而读取配置文件的信息),查找测试报告等等,经常会对大量文件和路径进行操作,这就需要依赖os模块。

1.2 os模块中常用的属性和函数

- 常用方法以及描述：

方法	描述
os.name	返回操作系统类型
os.environ	以字典形式返回系统变量
os.putenv(key, value)	改变或添加环境变量
os.listdir(path=' :)	列表形式列出目录下所有目录和文件名
os.getcwd()	获取当前路径
os.chdir(path)	改变当前工作目录到指定目录
os.mkdir(path [, mode $= 0777$])	创建目录
os.makedirs(path [mode=0777])	递归创建目录
os.rmdir(path)	移除空目录，不能删除有文件的目录
os.remove(path)	移除文件
os.rename(old, new)	重命名文件或目录
os.stat(path)	获取文件或目录属性
os.chown(path, uid, gid)	改变文件或目录所有者
os.chmod(path, mode)	改变文件访问权限
os.symlink(src, dst)	创建软链接
os.unlink(path)	移除软继接
os.getuid()	返回当前进程UID
os.getlogin()	返回登录用户名
os.getpid()	返回当前进程ID
os.kill(pid, sig)	发送一个信号给进程
os.walk(path)	目录树生成器，生成一个三组 (dirpath, dirnames, filenames)
os.system(command)	执行shell命令，不能存储结果
os.popen(command [, mode='r' [, bufsize]])	打开管道来自shell命令，并返回一个文岽对象

1.2.1 os.getcwd()

- `os.getcwd()`：函数
- 查看当前所在目录的路径：

```
>>> import os
>>> os.getcwd()   # pwd

'/Users/mengzhichao/Desktop/test'
```

1.2.2 os.chdir()

- `os.chdir()`：函数
- 切换目录

```
>>> os.chdir('/Users/mengzhichao/Desktop/test/src')
# cd /Users/mengzhichao/Desktop/test/src
>>> os.getcwd() # pwd

'/Users/mengzhichao/Desktop/test/src'
```

1.2.3 os.curdir、os.pardir

- `os.curdir`：属性

  `os.pardir`：属性
- 返回当前目录字符串名、返回当前目录的父目录的字符串名；

```
>>> os.curdir
'.'
>>> os.pardir
'..'
```

1.2.4 os.makedirs()

- `os.makedirs()`：函数
- 生成一个多层递归目录

```
>>> os.makedirs('dir_1/dir_2')
```

```
(base) mengzhichaodeMacBook-Pro:src mengzhichao$ tree ../src/
../src/
└── dir_1
    └── dir_2

2 directories, 0 files
```

1.2.5 os.removedirs()

- `os.removedirs()`：函数
- 若目录为空，则删除，并递归到上一级目录，如若也为空，则删除，依次类推；

```
>>> os.removedirs('dir_1/dir_2')
```

```
(base) mengzhichaodeMacBook-Pro:src mengzhichao$ tree ../src/
../src/

0 directories, 0 files
```

1.2.6 os.mkdir()

- `os.mkdir()`：函数
- 只能创建一个目录

```
>>> os.mkdir('test')
```

1.2.7 os.rmdir()

- `os.rmdir()`：函数
- 删除一个目录，若目录不为空则无法删除，报错

```
>>> os.rmdir('test')
```

1.2.8 os.listdir()

- `os.listdir()`：函数
- 显示指定目录下，所有的文件和子目录，包括隐藏文件；

```
>>> dir_list = os.listdir('/users')
>>> dir_list
['.localized', 'Shared', 'mengzhichao']
```

```
(base) mengzhichaodeMacBook-Pro:users mengzhichao$ ls -la
total 0
drwxr-xr-x   5 root         admin   160  4  2 17:10 .
drwxr-xr-x  31 root         wheel   992  6 11 14:25 ..
-rw-r--r--   1 root         wheel     0 10  7  2017 .localized
drwxrwxrwt  10 root         wheel   320  4  3 00:34 Shared
drwxr-xr-x+ 62 mengzhichao  staff  1984  7  2 14:28 mengzhichao
```

1.2.9 os.remove()

- `os.remove()`：函数
- 删除文件但不能删除文件夹

```
os.remove('test.txt')
```

1.2.10 os.stat()

- `os.stat()`：函数

- 获取文件/目录信息,并可以获取到文件的大小;

```
>>> info = os.stat('test.txt')
>>> print(info)
os.stat_result(st_mode=33188, st_ino=5764928, st_dev=16777220, st_nlink=1, st_uid=501, st_gid=20, st_size=98, st_atime=1625209429, st_mtime=1625209405, st_ctime=1625209405)
>>> info.st_size
98
```

1.2.11 os.sep

- `os.sep`：属性
- 输出操作系统指定的路径分隔符，如：windows 为'\',Linux为'/'；

```
>>> os.sep
'/'
```

1.2.12 os.pathsep

- `os.pathsep`：属性
- 输出用于分割文件路径的字符串；

```
>>> os.pathsep
':'
```

1.2.13 os.system()

- `os.system()`：函数
- 运行shell命令，直接显示（相当于启动一个全新的shell，然后去执行那条命令，命令执行完成过后，shell直接退出）；

```
>>> os.system('cat test.txt')
import pandas as pd
import numpy as np
for i in range(10):
  print('测试下，不必当真')
0
```

1.2.14 os.environ

- `os.environ`：属性
- 获取操作系统的环境变量;

```
>>> os.environ
environ({'MANPATH': '/opt/local/share/man:/usr/local/share/man:', 'TERM_PROGRAM': 'Apple_Terminal', 'HOMEBREW_CELLAR': '/usr/local/Cellar', '_': '/Users/mengzhichao/anaconda3/bin/python'})
```

1.2.15 os.rename()

- `os.rename()`： 方法
- 重命名文件

```
>>> import os
>>> os.getcwd()
'/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package/save'
>>> os.listdir()
['10、Package_argparse.png', 'wordcloud.jpg']

"""重命名文件"""
>>> os.rename('wordcloud.jpg','wordcloud.png')
>>> os.listdir()
['10、Package_argparse.png', 'wordcloud.png']
```

1.3 os.path相关

- 相关方法以及描述：

方法	描述
os.path.basename(path)	返回最后一个文件或目录名
os.path.dirname(path)	返回最后一个文件所属目录
os.path.abspath(path)	返回一个绝对路径
os.path.exists(path)	判断路径是否存在，返回布尔值
os.path.isdir(path)	判断是否是目录
os.path.isfile(path)	判断是否是文件
os.path.islink(path)	判断是否是链接
os.path.ismount(path)	判断是否挂载
os.path.getatime(filename)	返回文件访问时间截
os.path.getctime(filename)	返回文件变化时间戰
os.path.getmtime(filename)	返回文件修改时间戰
os.path.getsize(filename)	返回文件大小，单位字节
os.path.join(a, *p)	加入两个或两个以上路径，以正斜杜"/"分隔。常用于拼接路径
os.path.split(	分隔路径名
os.path.splitext(	分隔扩展名

1.3.1 os.path.split(path)

- `os.path.split(path)` 函数
- 将path分割成目录和文件名二元组返回;

```
>>> os.path.split('/Users/mengzhichao/Desktop/test/src/test.txt')
('/Users/mengzhichao/Desktop/test/src', 'test.txt')
```

1.3.2 os.path.abspath(path)

- `os.path.abspath(path)`：函数
- 返回path规范化的绝对路径; 函数

```
>>> os.path.abspath('test')
'/Users/mengzhichao/Desktop/test/src/test'
```

1.3.3 os.path.dirname(path)

- `os.path.dirname(path)`：函数
- 返回path的目录;

```
>>> os.path.dirname('/Users/mengzhichao/Desktop/test/src/test.txt')
'/Users/mengzhichao/Desktop/test/src'
```

1.3.4 os.path.basename(path)

- `os.path.basename(path)`：函数
- 返回path最后的文件名（一个绝对路径只返回最后的文件名）；

```
>>> os.path.basename('abc')
'abc'
>>> os.path.basename('/test/123/abc')
'abc'
```

1.3.5 os.path.exists(path)

- `os.path.exists(path)`：函数
- 判断路径是否存在，如果path存在，返回True；如果不存在，返回Flase;

```
>>> os.path.exists('/Users/mengzhichao/Desktop/test/src')
True
>>> os.path.exists('/Users/mengzhichao/Desktop/test/src/123')
False
```

1.3.6 os.path.isabs(path)

- `os.path.isabs(path)`：函数
- 判断是否是绝对路径，如果是，则返回True;

```
>>> os.path.isabs('./test.txt')
False
>>> os.path.isabs('/Users/mengzhichao/Desktop/test/src')
True
```

1.3.7 os.path.isfile(path)

- `os.path.isfile(path)`：函数
- 判断是否是一个文件，如果是则返回True，否则返回False;

```
>>> os.path.isfile('./test.txt')
True
>>> os.path.isfile('../test')
False
```

1.3.8 os.path.isdir(path)

- `os.path.isdir(path)`：函数
- 判断是否是一个存在的目录(注意是目录，传入已经存在的文件也会返回False)；

```
>>> os.path.isdir('/Users/mengzhichao/Desktop/test/src/test.txt')
False
>>> os.path.isdir('/Users/mengzhichao/Desktop/test/src')
True
```

1.3.9 os.path.join(path1， [path2],[path3])

- `os.path.join(path1, [path2],[path3])`：函数
- 将路径和文件名分为一个列表中的两个元素，将它们拼起来， 若其中有绝对路径,则之前的path将会被删除. ；

```
>>> os.path.join('/test/','hello.txt')
'/test/hello.txt'
>>> os.path.join('/test/123','hello.txt')
'/test/123/hello.txt'
```

1.3.10 os.path.getatime(path)

- `os.path.getatime(path)`：函数
- 返回path所指向的文件或者目录的 最后存取时间;

```
>>> os.path.getatime('/Users/mengzhichao/Desktop/test')
1625209180.4501557
```

1.3.11 os.path.getmtime(time)

- `os.path.getmtime(time)`：函数
- 返回path所指向的文件或者目录的 最后修改时间 ；

```
>>> os.path.getmtime('/Users/mengzhichao/Desktop/test')
1625207886.1682236
```

1.3.12 os.popen('ls')

- `os.popen('ls')`：函数
- 相当于打开了一个临时的文件存储打开的目录（可以赋给变量，字符串的形式)；

```
>>> os.chdir('/users/mengzhichao/')
>>> a = os.popen('ls -lha').read()
>>> print(a)
total 1323304
drwxr-xr-x+  62 mengzhichao  staff   1.9K  7  2 15:28 .
drwxr-xr-x    5 root         admin   160B  4  2 17:10 ..
-r--------    1 mengzhichao  staff     9B  4 20 11:48 .CFUserTextEncoding
-rw-r--r--@   1 mengzhichao  staff    22K  7  1 17:40 .DS_Store
drwx------    8 mengzhichao  staff   256B  7  2 14:52 .Trash
drwxr-xr-x    3 mengzhichao  staff    96B  4 29 16:21 .anaconda
drwxr-xr-x    3 mengzhichao  staff    96B  4  2 21:12 .astropy

eg...
```

1.3.13 os.path.splitext()

- `os.path.splitext()`:函数
- 可以用来获取文件后缀

```
>>> os.path.splitext('/tmp/a.txt')
('/tmp/a', '.txt')
>>> os.path.splitext('/tmp/a.txt')[1]
'.txt'
```

1.4 遍历目录

```
>>> for dirpath,dirname,filename in os.walk('/tmp'):
...     print(f"目录路径:{dirpath},目录名:{dirname},文件名:{filename}")
...
目录路径:/tmp,目录名:['com.google.Keystone', 'powerlog', 'com.sogou.inputmethod', 'com.apple.launchd.B99za8vbj2', 'com.apple.launchd.h3hyNRLCcO'],文件名:[]
目录路径:/tmp/com.google.Keystone,目录名:[],文件名:['.keystone_system_install_lock', '.keystone_install_lock']
目录路径:/tmp/powerlog,目录名:[],文件名:[]
目录路径:/tmp/com.sogou.inputmethod,目录名:[],文件名:[]
目录路径:/tmp/com.apple.launchd.B99za8vbj2,目录名:[],文件名:['Listeners']
目录路径:/tmp/com.apple.launchd.h3hyNRLCcO,目录名:[],文件名:['Render']

```

2 二、 argparse 模块

2.1 argparse模块简介

- argparse是Python中的一个常用模块，和sys.argv()功能类似，主要用于编写命令行接口：对于程序所需要的参数，它可以进行正确的解析。另外，argparse还可以自动的生成help和usage信息，当程序的参数无效时，它可以自动生成错误信息。

- **官方文档**

> https://docs.python.org/3/library/argparse.html

2.2 argparse例子

- 下列代码是python程序，它可以接受多个整数，并返回它们的 `和` 或者 `最大值` 。

```python
import argparse

parser = argparse.ArgumentParser(description='处理一些整数')
parser.add_argument('integers', metavar='N', type=int, nargs='+',
                    help='累加器中的整数')
parser.add_argument('--sum', dest='accumulate', action='store_const',
                    const=sum, default=max,
                    help='对整数进行求和(默认: 寻找最大值)')
args = parser.parse_args()
print(args.accumulate(args.integers))
```

假定上述代码保存为 `test.py` ，它可以在命令行执行并自动提供有用的 `help` 信息：

```
(base) mengzhichaodeMacBook-Pro:Sample_Code mengzhichao$ python test.py -h
usage: test.py [-h] [--sum] N [N ...]

处理一些整数

positional arguments:
  N           累加器中的整数

optional arguments:
  -h, --help  show this help message and exit
  --sum       对整数进行求和(默认：寻找最大值)

```

该程序可以接受相应的参数并给出相应的输出：

```
(base) mengzhichaodeMacBook-Pro:Sample_Code mengzhichao$ python test.py 1 2 3 4
4
(base) mengzhichaodeMacBook-Pro:Sample_Code mengzhichao$ python test.py 1 2 3 4 --sum
10
```

```

  如果传入无效的参数，它可以自动生成`error`信息：

  ```bash
  (base) mengzhichaodeMacBook-Pro:Sample_Code mengzhichao$ python test.py  a b
  usage: test.py [-h] [--sum] N [N ...]
test.py: error: argument N: invalid int value: 'a'
```

2.3 argparse 的三个主要函数

2.3.1 **Creating a parser**：

- `parser=argparse.ArgumentParser()`

  使用argparse的第一步是创建ArgumentParser对象:

```
parser = argparse.ArgumentParser(description='处理一些整数')
```

ArgumentParser对象保存了所有必要的信息，用以将命令行参数解析为相应的`python`数据类型。
```

2.3.2 Adding arguments：

- `parser.add_argument()`

  调用add_argument() 向ArgumentParser对象添加命令行参数信息，这些信息告诉ArgumentParser对象如何处理命令行参数。可以通过调用parse_agrs()来使用这些命令行参数。例如：

```
parser.add_argument('integers', metavar='N', type=int, nargs='+',
                    help='累加器中的整数')
parser.add_argument('--sum', dest='accumulate', action='store_const',
                    const=sum, default=max,
                    help='对整数进行求和(默认：寻找最大值)')
```

  之后，调用parse.args()将返回一个对象，它具有两个属性：`integers` 和 `accumulate`，前者是一个整数集合，后者根据命令行参数的不同，代表不同的函数，可以是max()，也可以是sum()。

2.3.3 Parsing arguments：

- `args = parser.parse_args()`

  通过调用parse_args()来解析ArgumentParser对象中保存的命令行参数：将命令行参数解析成相应的数据类型并采取相应的动作，它返回一个 `Namespace` 对象。

```
>>> parser.parse_args(['--sum', '7', '-1', '42'])
Namespace(accumulate=<built-in function sum>, integers=[7, -1, 42])
```

  在实际的 `python` 脚本中，parse_args()一般并不使用参数，它的参数由 `sys.argv` 决定。

2.4 ArgumentParser对象(ArgumentParser objects)

```
class argparse.ArgumentParser(prog=None
                              , usage=None
                              , description=None
                              , epilog=None
                              , parents=[]
                              , formatter_class=argparse.HelpFormatter
                              , argument_default=None
                              , conflict_handler='error'
                              , add_help=True
                              , allow_abbrev=True
                              )
```

2.4.1 对象参数简介

- 创建一个新的 ArgumentParser 对象。所有参数都应作为关键字参数传递。每个参数在下面都有更详细的描述，但简而言之，它们是：

- **prog** - The name of the program (default: `sys.argv[0]`)
    - 程序文件名
- **usage** - The string describing the program usage (default: generated from arguments added to parser)
    - 程序使用说明
- **description** - Text to display before the argument help (default: none)
    - 程序目的说明
- **epilog** - Text to display after the argument help (default: none)
    - 程序说明后记
- **parents** - A list of `ArgumentParser` objects whose arguments should also be included
    - ArgumentParser对象的父对象的参数列表。
- **formatter_class** - A class for customizing the help output
    - `help`信息的说明格式
- **prefix_chars** - The set of characters that prefix optional arguments (default: '-')
    - 命令行参数前缀。
- **fromfile_prefix_chars** - The set of characters that prefix files from which additional arguments should be read (default: `None`)
- **argument_default** - The global default value for arguments (default: `None`)
    - 参数全局默认值
- **conflict_handler** - The strategy for resolving conflicting optionals (usually unnecessary)
    - 冲突处理
- **add_help** - Add a `-h/--help` option to the parser (default: `True`)
    - 是否增加help选项
- **allow_abbrev** - Allows long options to be abbreviated if the abbreviation is unambiguous. (default: `True`)
    - 是否使用参数缩写
- **exit_on_error** - Determines whether or not ArgumentParser exits with error info when an error occurs. (default: `True`)

2.4.2 常用对象参数具体用法

- `prog`(程序文件名)：

  默认情况下，ArgumentParser对象使用sys.argv[0]来决定如何在`help`信息中显示程序的文件名。
  例如：下列代码的文件名为myprogram.py

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument('--foo', help='foo help')
args = parser.parse_args()
```

它的 `help` 信息将显示程序的文件名，而不管程序在何处进行调用。

```
(base) mengzhichaodeMacBook-Pro:Sample_Code mengzhichao$ python myprogram.py  --help
usage: myprogram.py [-h] [--foo FOO]

optional arguments:
  -h, --help  show this help message and exit
  --foo FOO   foo help
(base) mengzhichaodeMacBook-Pro:Sample_Code mengzhichao$ cd ..
(base) mengzhichaodeMacBook-Pro:16、python代码 mengzhichao$ python Sample_Code/myprogram.py --help
usage: myprogram.py [-h] [--foo FOO]

optional arguments:
  -h, --help  show this help message and exit
  --foo FOO   foo help
```

使用prog参数可以修改ArgumentParser对象的默认文件名：

```
$parser = argparse.ArgumentParser(prog='myprogram')
$parser.print_help()
usage: myprogram [-h]

optional arguments:
 -h, --help  show this help message and exit
```

不论程序的文件名是来自sys.argv[0]，不是来自参数prog，都可以通过%(prog)s来引用程序的文件名。

```
$ parser = argparse.ArgumentParser(prog='myprogram')
$ parser.add_argument('--foo', help='foo of the %(prog)s program')
$ parser.print_help()
usage: myprogram [-h] [--foo FOO]

optional arguments:
 -h, --help  show this help message and exit
 --foo FOO   foo of the myprogram program
```

- `usage` 程序使用说明

  默认情况下，ArgumentParser对象会根据它所包括的参数来自动的生成使用说明。

```
$ parser = argparse.ArgumentParser(prog='PROG')
$ parser.add_argument('--foo', nargs='?', help='foo help')
$ parser.add_argument('bar', nargs='+', help='bar help')
$ parser.print_help()
usage: PROG [-h] [--foo [FOO]] bar [bar ...]

positional arguments:
 bar           bar help

optional arguments:
 -h, --help    show this help message and exit
 --foo [FOO]   foo help
```

可以使用参数usage来修改程序的使用说明信息。

```
$ parser = argparse.ArgumentParser(prog='PROG', usage='%(prog)s [options]')
$ parser.add_argument('--foo', nargs='?', help='foo help')
$ parser.add_argument('bar', nargs='+', help='bar help')
$ parser.print_help()
usage: PROG [options]

positional arguments:
 bar           bar help

optional arguments:
 -h, --help    show this help message and exit
 --foo [FOO]   foo help
```

- `description` 程序目的说明

  一般情况下，ArgumentParser对象会使用参数description来说明程序的用途以及目的。`description` 信息一般位于 `usage` 信息和 `help` 信息之间

```
$ parser = argparse.ArgumentParser(description='A foo that bars')
$ parser.print_help()
usage: argparse.py [-h]

A foo that bars

optional arguments:
 -h, --help  show this help message and exit
```

  默认情况下， `description` 会自动调整间距来适应显示空间。可以通过 `formatter_class` 类来修改它的显示方式。

- epilog 程序说明后记

  一些程序喜欢在参数描述信息之后添加额外的信息，这些信息可以通过参数epilog来指定。

```
$  parser = argparse.ArgumentParser(
...      description='A foo that bars',
...      epilog="And that's how you'd foo a bar")
$ parser.print_help()
usage: argparse.py [-h]

A foo that bars

optional arguments:
 -h, --help  show this help message and exit

And that's how you'd foo a bar
```

和 `description` 类似， `epilog` 会自动调整间距来适应显示空间。可以通过 `formatter_class` 类来修改它的显示方式。

2.5 add_argument()方法

2.5.1 函数参数简介

- 函数参数
  - **name or flags** - Either a name or a list of option strings, e.g. `foo` or `-f, --foo`.
    - 参数名或者参数标识
    - 带 `-` 的为可选参数 （optional parameter）
    - 不带 `-` 的为必选参数 （positional parametrer） 。
  - **action** - The basic type of action to be taken when this argument is encountered at the command line.
    - 参数的处理方法
  - **nargs** - The number of command-line arguments that should be consumed.
    - 参数的数量
  - **const** - A constant value required by some **action** and **nargs** selections.
    - 参数的常量值
  - **default** - The value produced if the argument is absent from the command line and if it is absent from the namespace object.
    - 参数默认值
  - **type** - The type to which the command-line argument should be converted.
    - 参数的数据类型
  - **choices** - A container of the allowable values for the argument.
    - 参数取值范围
  - **required** - Whether or not the command-line option may be omitted (optionals only).

        - 参数是否可以忽略不写， 仅对可选参数有效
    - **help** - A brief description of what the argument does.
        - 参数的说明信息
    - **metavar** - A name for the argument in usage messages.
        - 参数在说明信息 `usage` 中的名称
    - **dest** - The name of the attribute to be added to the object returned by `parse_args()` .
        - 对象的属性名

2.5.2 常用函数参数具体用法

- `default`： 默认参数值

```
"""程序名称: myprogram.py"""
import argparse

def get_parser():
    parser = argparse.ArgumentParser(description="Demo of argparse")
    parser.add_argument('--name'
                        , default='Great'    # 默认值为Great
                       )

    return parser

if __name__ == '__main__':
    parser = get_parser()
    args = parser.parse_args()
    name = args.name
    print('Hello {}'.format(name))

```

终端运行结果

```
$ python myprogram.py
Hello Great
$ python myprogram.py  --name mengzhichao
Hello mengzhichao
```

- `required`： 表示这个参数是否一定要设置

如果设置了 `required=True` ,则在实际运行的时候不设置该参数将报错（即使在你已经指定了default的情况下）：

```
$ python myprogram.py
usage: myprogram.py [-h] --name NAME
myprogram.py: error: the following arguments are required: --name
$ python myprogram.py --name mengzhichao
Hello mengzhichao
```

- `type`： 参数类型

  默认的参数类型是str类型，如果你的程序需要一个整数或者布尔型参数，你需要设置 `type=int` 或 `type=bool`， 下面是一个打印平方的例子：

```python
"""程序名称: myprogram.py"""
import argparse


def get_parser():
    parser = argparse.ArgumentParser(
        description='Calculate square of a given number')
    parser.add_argument('-number'
                        , type=int # 参数类型
                        , required=True # 该参数是否一定需要
                        , default=10 # 参数默认值
                       )

    return parser


if __name__ == '__main__':
    parser = get_parser()
    args = parser.parse_args()
    res = args.number ** 2
    print('square of {} is {}'.format(args.number, res))
```

  如果不是指定的参数类型，那么会报错

```
$ python myprogram.py -number 10
square of 10 is 100
$ python myprogram.py -number 'str'
usage: myprogram.py [-h] -number NUMBER
myprogram.py: error: argument -number: invalid int value: 'str'
```

- `choices`： 参数值只能从几个选项里面选择

```python
import argparse


def get_parser():
    parser = argparse.ArgumentParser(
        description='choices demo')
    parser.add_argument('-arch'
                        , required=True
                        , choices=['alexnet', 'vgg']
                        , type=str)

    return parser


if __name__ == '__main__':
```

```
    parser = get_parser()
    args = parser.parse_args()
    print('the arch of CNN is {}'.format(args.arch))
```

如果向下面第一条这样执行会报错，因为第一条所给的 `-arch` 参数 `resnet` 不在备选的 `choices` 之中，所以会报错：

```
$ python myprogram.py -arch resnet
usage: myprogram.py [-h] -arch {alexnet,vgg}
myprogram.py: error: argument -arch: invalid choice: 'resnet' (choose from 'alexnet', 'vgg')

$ python myprogram.py -arch alexnet
the arch of CNN is alexnet
```

- `help`：指定参数的说明信息

```
import argparse

def get_parser():
    parser = argparse.ArgumentParser(
        description='help demo')
    parser.add_argument('-arch'
                        , required=True
                        , choices=['alexnet', 'vgg']
                        , help='the architecture of CNN, \
                        at this time we only support alexnet and vgg.')


    return parser


if __name__ == '__main__':
    parser = get_parser()
    args = parser.parse_args()

    print('the arch of CNN is {}'.format(args.arch))
```

在命令行加 `-h` 或 `--help` 参数运行该命令，获取帮助信息的时候，结果如下：

```
$ python myprogram.py --help
usage: myprogram.py [-h] -arch {alexnet,vgg}

help demo

optional arguments:
  -h, --help            show this help message and exit
  -arch {alexnet,vgg}   the architecture of CNN, at this time we only support
                        alexnet and vgg.

```

- `dest`：设置参数在代码中的别名---被添加到 `parse_args()` 所返回对象上的属性名：
  注意：为该参数设置别名后，只能用别名在namespace中调用。（参考下述代码8　22 行）

```
1   import argparse
2
3
4   def get_parser():
5       parser = argparse.ArgumentParser(
6           description='help demo')
7       parser.add_argument('-arch'
8                           , dest='ar'
9                           , required=False
10                          , default='alexnet'
11                          , choices=['alexnet', 'vgg']
12                          , help='the architecture of CNN, \
13                          at this time we only support alexnet and
    vgg.')
14
15      return parser
16
17
18  if __name__ == '__main__':
19      parser = get_parser()
20      args = parser.parse_args()
21
22      print('the arch of CNN is {}'.format(args.ar))
```

- `nargs`：设置参数在使用时可以提供的个数
  使用方式如下

> parser.add_argument('-name', nargs=x)

其中 `x` 的候选值和含义如下：

值	含义
N	参数的绝对个数（如：3）
?	0或者1个参数
*	0或所有参数
+	所有，并且至少一个参数

如下例子：

```
1   import argparse
2
```

```python

def get_parser():
    parser = argparse.ArgumentParser(
        description='nargs demo')
    parser.add_argument('-name'
                        , type=str  # 类型
                        , dest='nas'  # 别名
                        , metavar='names'  # 在usage中显示的名称
                        , required=False  # 是否一定需要
                        , default='A'  # 默认
                        , choices=['A', 'B', 'C'] # 参数选择范围
                        , nargs='+'
                        , )

    return parser


if __name__ == '__main__':
    parser = get_parser()
    args = parser.parse_args()
    names = ', '.join(args.nas)
    print('Hello to {}'.format(names))

```

执行命令和结果如下：

```
$ python myprogram.py -name A B C
Hello to A, B, C

$ python myprogram.py -name A g
usage: myprogram.py [-h] [-name names [names ...]]
myprogram.py: error: argument -name: invalid choice: 'g' (choose from 'A', 'B', 'C')
```

2.6 parse_args()方法

```
ArgumentParser.parse_args(args=None, namespace=None)
```

`parse_args()` 方法将命令行参数字符串转换为相应对象并赋值给 `Namespace` 对象的相应属性，默认返回一个 `Namespace` 对象。

1. `args`：字符串列表，默认来自sys.argv。
2. `namespace`:对象名，默认是一个空的namespace对象。

2.7 namespace 对象

```
class argparse.Namespace
```

调用parse_args()的返回值是一个 `Namespace` 对象，它具有很多属性，每个属性都代表相应的命令行参数。

`Namespace` 对象是一个非常简单的类，可以通过vars()将之转换成字典类型。例如：

```
>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--test')
_StoreAction(option_strings=['--test'], dest='test', nargs=None, const=None, default=None, type=None, choices=None, help=None, metavar=None)
>>> args = parser.parse_args(['--test', 'TEST'])
>>> args
Namespace(test='TEST')
>>> vars(args)
{'test': 'TEST'}
```

- 另外还可以将ArgumentParser对象赋值给别的命令空间，而不是新建一个 `Namespace` 对象，例如：

```
$ class C:
...     pass
...
$ c = C()
$ parser = argparse.ArgumentParser()
$ parser.add_argument('--foo')
$ parser.parse_args(args=['--foo', 'BAR'], namespace=c)
$ c.foo
'BAR'
```

2.8 argparse 实例（以决策树参数、随机森林参数为例）

2.8.1 直接添加

- 主体

```
def get_parser():
    """
    为决策树算法提供参数
    :return: 参数字典
    """
    parser = argparse.ArgumentParser(
        description='为决策树算法提供参数'  # 程序目的说明
    )
    parser.add_argument('--criterion'
                        , required=False
                        , default='entropy'
```

```
                                , choices=['gini', 'entropy']
                                , type=str
                                , help='决策树节点分裂准则选择')
        parser.add_argument('--splitter'
                                , required=False
                                , default='best'
                                , choices=['best', 'random']
                                , type=str
                                , help='决策树随机/最优选择特征划分')
        parser.add_argument('--max_depth'
                                , required=False
                                , default=None
                                , type=int
                                , help='决策树最大深度')
        parser.add_argument('--min_samples_split'
                                , required=False
                                , default=2
                                , type=int
                                , help='决策树父节点最少划分样本量')
        parser.add_argument('--min_samples_leaf'
                                , required=False
                                , default=2
                                , type=int
                                , help='决策树叶子节点最少划分样本量')
        parser.add_argument('--min_weight_fraction_leaf'
                                , required=False
                                , default=0
                                , type=int
                                , help='决策树忘了这是啥')
        parser.add_argument('--max_leaf_nodes'
                                , required=False
                                , default=None
                                , type=int
                                , help='决策树最大叶子节点数')
        parser.add_argument('--random_state'
                                , required=False
                                , default=10
                                , type=int
                                , help='随机模式参数')
        parser.add_argument('--max_features'
                                , required=False
                                , default=None
                                , type=int
                                , help='决策树训练用最大特征数量')
        parser = parser.parse_args()
        print(vars(parser))
        return vars(parser)
```

- 查看help

```
$ python 10、Package_argparse.py --help
usage: 10、Package_argparse.py [-h] [--criterion {gini,entropy}]
```

```
                                [--splitter {best,random}]
                                [--max_depth MAX_DEPTH]
                                [--min_samples_split MIN_SAMPLES_SPLIT]
                                [--min_samples_leaf MIN_SAMPLES_LEAF]
                                [--min_weight_fraction_leaf MIN_WEIGHT_FRACTION_LEAF]
                                [--max_leaf_nodes MAX_LEAF_NODES]
                                [--random_state RANDOM_STATE]
                                [--max_features MAX_FEATURES]

为决策树算法提供参数

optional arguments:
  -h, --help            show this help message and exit
  --criterion {gini,entropy}
                        决策树节点分裂准则选择
  --splitter {best,random}
                        决策树随机/最优选择特征划分
  --max_depth MAX_DEPTH
                        决策树最大深度
  --min_samples_split MIN_SAMPLES_SPLIT
                        决策树父节点最少划分样本量
  --min_samples_leaf MIN_SAMPLES_LEAF
                        决策树叶子节点最少划分样本量
  --min_weight_fraction_leaf MIN_WEIGHT_FRACTION_LEAF
                        决策树忘了这是啥
  --max_leaf_nodes MAX_LEAF_NODES
                        决策树最大叶子节点数
  --random_state RANDOM_STATE
                        随机模式参数
  --max_features MAX_FEATURES
                        决策树训练用最大特征数量

```

- 全部代码

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 10、Package_argparse.py
@time: 2021-07-06 09:23
**************************************************
"""
import argparse
from sklearn.datasets import load_wine, load_breast_cancer
```

```python
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from sklearn.metrics import plot_roc_curve, plot_precision_recall_curve, plot_confusion_matrix
from sklearn.tree import DecisionTreeClassifier
import pandas as pd
from matplotlib import pyplot as plt
import os
import sys

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams["font.family"] = 'Arial Unicode MS'

global cur_file_path
cur_file_path = os.path.split(sys.argv[0])[0]


def get_parser():
    """
    为决策树算法提供参数
    :return: 参数字典
    """
    parser = argparse.ArgumentParser(
        description='为决策树算法提供参数'  # 程序目的说明
    )
    parser.add_argument('--criterion'
                        , required=False
                        , default='entropy'
                        , choices=['gini', 'entropy']
                        , type=str
                        , help='决策树节点分裂准则选择')
    parser.add_argument('--splitter'
                        , required=False
                        , default='best'
                        , choices=['best', 'random']
                        , type=str
                        , help='决策树随机/最优选择特征划分')
    parser.add_argument('--max_depth'
                        , required=False
                        , default=None
                        , type=int
                        , help='决策树最大深度')
    parser.add_argument('--min_samples_split'
                        , required=False
                        , default=2
                        , type=int
                        , help='决策树父节点最少划分样本量')
    parser.add_argument('--min_samples_leaf'
                        , required=False
                        , default=2
                        , type=int
```

```
                                , help='决策树叶子节点最少划分样本量')
        parser.add_argument('--min_weight_fraction_leaf'
                                , required=False
                                , default=0
                                , type=int
                                , help='决策树忘了这是啥')
        parser.add_argument('--max_leaf_nodes'
                                , required=False
                                , default=None
                                , type=int
                                , help='决策树最大叶子节点数')
        parser.add_argument('--random_state'
                                , required=False
                                , default=10
                                , type=int
                                , help='随机模式参数')
        parser.add_argument('--max_features'
                                , required=False
                                , default=None
                                , type=int
                                , help='决策树训练用最大特征数量')
        parser = parser.parse_args()
        print(vars(parser))
        return vars(parser)


class DT:
    def __init__(self, criterion, splitter, max_depth
                    , min_samples_split, min_samples_leaf
                    , min_weight_fraction_leaf, max_leaf_nodes
                    , random_state, max_features, data=None
                    , X=None
                    , y=None):
        """
        初始化函数
        :param criterion: 决策树节点分裂准则选择
        :param splitter: 决策树随机/最优选择特征划分
        :param max_depth: 决策树最大深度
        :param min_samples_split: 决策树父节点最少划分样本量
        :param min_samples_leaf: 决策树叶子节点最少划分样本量
        :param min_weight_fraction_leaf: 决策树忘了这是啥
        :param max_leaf_nodes: 决策树最大叶子节点数
        :param random_state: 随机模式参数
        :param max_features: 决策树训练用最大特征数量
        :param data: 数据集
        :param X: X
        :param y: y
        """
        self.criterion = parameters[criterion]
        self.splitter = parameters[splitter]
        self.max_depth = parameters[max_depth]
        self.min_samples_split = parameters[min_samples_split]
```

```python
        self.min_samples_leaf = parameters[min_samples_leaf]
        self.min_weight_fraction_leaf = parameters[min_weight_fraction_leaf]
        self.max_leaf_nodes = parameters[max_leaf_nodes]
        self.random_state = parameters[random_state]
        self.max_features = parameters[max_features]

        self.data = data  # 传进来的data包含标签列（y）
        self.feature_names = self.data.columns[:-1] if self.data is not None else None
        self.X = X
        self.y = y

    def create_datas(self):
        '''创建数据集'''
        wine = load_wine()
        self.X = wine.data
        self.y = wine.target
        self.feature_names = wine.feature_names

        self.data = pd.DataFrame(self.X, columns=self.feature_names)
        self.data['target'] = pd.DataFrame(self.y)

    def data_split(self):
        """训练集、测试集划分"""
        return train_test_split(self.X
                                , self.y
                                , test_size=0.3
                                , random_state=self.random_state)

    def Modeling(self, X_train, X_test, y_train, y_test):
        """
        训练决策树
        """
        clf = DecisionTreeClassifier(criterion=self.criterion
                                     , splitter=self.splitter
                                     , max_depth=self.max_depth
                                     , min_samples_split=self.min_samples_split
                                     , min_samples_leaf=self.min_samples_leaf
                                     , min_weight_fraction_leaf=self.min_weight_fraction_leaf
                                     , max_leaf_nodes=self.max_leaf_nodes
                                     , random_state=self.random_state
                                     , max_features=self.max_features
                                     )
        clf.fit(X_train, y_train)
```

```python
        score = clf.score(X_test, y_test)  # accuracy
        print(f'decision tree accuracy:{score}')

        y_test_predict = clf.predict(X_test)  # 测试集预测结果
        report_of_classifier = classification_report(y_test, y_test_predict)
        print(report_of_classifier)

        try:
            # roc曲线 and p_r曲线 and 混淆矩阵

            figure = plt.figure(num=1, figsize=(12, 8), dpi=300)
            ax1 = plt.subplot(212)
            plot_confusion_matrix(clf, X_test, y_test, ax=ax1)
            ax1.set_title('混淆矩阵')
            ax1.margins(2, 2)

            ax2 = plt.subplot(221)
            plot_roc_curve(clf, X_test, y_test, ax=ax2)
            ax2.set_title('ROC 曲线')

            ax3 = plt.subplot(222)
            plot_precision_recall_curve(clf, X_test, y_test, ax=ax3)
            ax3.set_title('P_R 曲线')

            # 当前程序文件名称（去除.py）
            cur_file_name = os.path.split(sys.argv[0])[1].split('.')[0]
            if not os.path.exists('save'):
                os.mkdir('save')

            # 图片存储路径
            photo_save_path = os.path.join(cur_file_path, 'save', cur_file_name + '.png')

            figure.savefig(photo_save_path)
            plt.show()

        except ValueError:
            print('DecisionTreeClassifier should be a binary classifier')

    def run(self):
        if self.data is None:
            self.create_datas()
        else:
            self.X = self.data.iloc[:, :-1]
            self.y = self.data.iloc[:, -1]

        X_train, X_test, y_train, y_test = self.data_split()
        self.Modeling(X_train, X_test, y_train, y_test)
```

```python


if __name__ == '__main__':
    # 创建乳腺癌数据集
    breast = load_breast_cancer()
    data = pd.DataFrame(breast.data, columns=breast.feature_names)
    data['target'] = pd.DataFrame(breast.target)

    parameters = get_parser()
    dt = DT(*parameters, data)
    # dt = DT(*parameters)

    dt.run()

```

- 可以直接运行，因为参数全都非必须，且都有默认值

```
$ python 10、Package_argparse.py
{'criterion': 'entropy', 'splitter': 'best', 'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 2, 'min_weight_fraction_leaf': 0, 'max_leaf_nodes': None, 'random_state': 10, 'max_features': None}
decision tree accuracy:0.9532163742690059
              precision    recall  f1-score   support

           0       0.89      0.98      0.94        59
           1       0.99      0.94      0.96       112

    accuracy                           0.95       171
   macro avg       0.94      0.96      0.95       171
weighted avg       0.96      0.95      0.95       171

```

也可以命令行指定参数运行

```
$ python 10、Package_argparse.py --criterion gini
{'criterion': 'gini', 'splitter': 'best', 'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 2, 'min_weight_fraction_leaf': 0, 'max_leaf_nodes': None, 'random_state': 10, 'max_features': None}
decision tree accuracy:0.9298245614035088
              precision    recall  f1-score   support

           0       0.87      0.93      0.90        59
           1       0.96      0.93      0.95       112

    accuracy                           0.93       171
   macro avg       0.92      0.93      0.92       171
weighted avg       0.93      0.93      0.93       171

```

2.8.2 通过配置文件（以随机森林为例，两个参数做测试）

- 主体：

配置文件

```
n_estimators:10
criterion:gini
```

主体代码

将配置文件中的参数读取，设置为默认（defualt），所以也可以手动改参数

```
def read_conf(cls, conf_name):
    """"""
    conf_items = {}
    with open(conf_name) as f:
        line = f.readline()  # 单行读取
        while line:
            # 忽略空行和注视
            if len(line.strip()) > 3 and (not line.strip().startswith('#')):
                kv_list = line.split(':')
                key = kv_list[0].strip()
                value = kv_list[1].strip()
                conf_items[key] = value
            line = f.readline()
    return conf_items


def get_parser():
    """
    usage : 获取参数信息

    process_path : 当前程序所在目录
    conf_info_path : 参数配置文件地址
    conf_dict : 处理完成返回的参数字典

    :return: 参数字典
    """

    conf_info_path = os.path.join(process_path, 'need/randomforecast.conf')

    conf_dict = read_conf(conf_info_path)

    parse = argparse.ArgumentParser(description='为随机隋林算法提供参数')
    parse.add_argument('--n_estimators'
                       , required=False
                       , type=int
                       , default=conf_dict['n_estimators']
                       , help='随机森林 基分类器数量')
    parse.add_argument('--criterion'
                       , required=False
```

```
                        , type=str
                        , default=conf_dict['criterion']
                        , choices=['entropy', 'gini']
                        , help='随机森林 基分类器节点分裂准则')

    parse = parse.parse_args()

    parameters = vars(parse)
    print(parameters)
    return parameters

```

- 全部代码

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: GeneralTools.py
@time: 2021-07-07 09:52
**************************************************
"""


class GeneralTools:
    @classmethod
    def read_conf(cls, conf_name):
        """"""
        conf_items = {}
        with open(conf_name) as f:
            line = f.readline()  # 单行读取
            while line:
                # 忽略空行和注视
                if len(line.strip()) > 3 and (not line.strip().startswith('#')):
                    kv_list = line.split(':')
                    key = kv_list[0].strip()
                    value = kv_list[1].strip()
                    conf_items[key] = value
                line = f.readline()
        return conf_items

```

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 2、RandomForecastClassifier.py
```

```
@time: 2021-07-07 09:50
**************************************************
"""

import os
import sys
import argparse
import pandas as pd
from GeneralTools import GeneralTools
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report
from matplotlib import pyplot as plt
from sklearn.metrics import plot_confusion_matrix, plot_precision_recall_curve, plot_roc_curve, plot_det_curve

global process_path
process_path = os.path.split(sys.argv[0])[0]

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams["font.family"] = 'Arial Unicode MS'


def get_parser():
    """
    usage : 获取参数信息

    process_path : 当前程序所在目录
    conf_info_path : 参数配置文件地址
    conf_dict : 处理完成返回的参数字典

    :return: 参数字典
    """

    conf_info_path = os.path.join(process_path, 'need/randomforecast.conf')

    conf_dict = GeneralTools().read_conf(conf_info_path)

    parse = argparse.ArgumentParser(description='为随机隋林算法提供参数')
    parse.add_argument('--n_estimators'
                       , required=False
                       , type=int
                       , default=conf_dict['n_estimators']
                       , help='随机森林 基分类器数量')
    parse.add_argument('--criterion'
                       , required=False
                       , type=str
```

```
                        , default=conf_dict['criterion']
                        , choices=['entropy', 'gini']
                        , help='随机森林 基分类器节点分裂准则')

    parse = parse.parse_args()

    parameters = vars(parse)
    print(parameters)
    return parameters


class RF:
    def __init__(self, n_estimators, criterion, data=None):
        self.criterion = parameters[criterion]
        self.n_estimators = parameters[n_estimators]

        self.data = data
        self.feature_names = self.data.columns if self.data is not None else None
        self.X = None
        self.y = None

    def transform_target(self, target):
        """仅对疫情target字段进行处理"""
        if target >= -0.2096:
            return '免疫力等级高'
        return '免疫力等级低'

    def data_preprocessing(self):
        """仅对疫情数据集进行处理"""
        self.data.drop(['企业名称', '分类', '企业性质_2'], axis=1, inplace=True)
        self.data.columns = ['industry', 'iso_num', 'qiye_nature', 'LC_performance',
                             'shareholder_num', 'shareholder_com', 'zl_zscq', 'ywfb',
                             'clsj', 'zczj', 'sjzj', 'person_num', 'difference']
        self.data['target'] = data['difference'].apply(self.transform_target)
        self.data.drop('difference', axis=1)

        le = LabelEncoder()
        data['target'] = le.fit_transform(data['target'])
        modeling_data = data.join(pd.get_dummies(data['shareholder_com'])).join(pd.get_dummies(data['industry'])).join(
            pd.get_dummies(data['qiye_nature'])).drop(['industry', 'qiye_nature', 'shareholder_com']
                                                      , axis=1)
        target_data = modeling_data['target']
        modeling_data.drop('target', axis=1, inplace=True)
```

```python
        modeling_data = modeling_data.join(target_data)

        self.data = modeling_data
        self.X = self.data.iloc[:, :-1]
        self.y = self.data.iloc[:, -1]

    def split_data(self):
        return train_test_split(self.X
                                , self.y
                                , random_state=120
                                , test_size=0.3)

    def create_data(self):
        breast = load_breast_cancer()
        self.feature_names = breast.feature_names
        self.data = pd.DataFrame(breast.data, columns=self.feature_names)
        self.data['target'] = pd.Series(breast.target)

        self.X = breast.data
        self.y = breast.target

    def modelling(self, X_train, X_test, y_train, y_test):
        rf_clf = RandomForestClassifier(n_estimators=self.n_estimators
                                        , criterion=self.criterion)
        rf_clf.fit(X_train, y_train)
        acc = rf_clf.score(X_test, y_test)

        print(f"accuracy:{acc}")

        y_test_pred = rf_clf.predict(X_test)

        report_result = classification_report(y_test, y_test_pred)
        print(report_result)

        try:
            fig, axes = plt.subplots(2, 2)
            fig.set_size_inches(20, 12)
            fig.set_dpi(300)

            plot_roc_curve(rf_clf, X_test, y_test, ax=axes[0, 0])
            axes[0, 0].set_title('roc 曲线')
            plot_precision_recall_curve(rf_clf, X_test, y_test, ax=axes[0, 1])
            axes[0, 1].set_title('P_R 曲线')
            plot_confusion_matrix(rf_clf, X_test, y_test, ax=axes[1, 0])
            axes[1, 0].set_title('混淆矩阵')
            plot_det_curve(rf_clf, X_test, y_test, ax=axes[1, 1])
            axes[1, 1].set_title('det 曲线')
```

```python

            plt.show()
        except:
            pass

    def run(self):
        if self.data is None:
            self.create_data()
        else:
            self.data_preprocessing()

        X_train, X_test, y_train, y_test = self.split_data()

        self.modelling(X_train, X_test, y_train, y_test)


if __name__ == '__main__':
    parameters = get_parser()

    data = pd.read_excel(os.path.join(process_path, 'need/疫情数据集.xlsx'))

    rf = RF(*parameters, data)
    rf.run()
```

2.9 封装

- 通用类

  增加参数在 `initialize` 函数中增加

```python
import argparse
import logging

class BaseOptions:
    """BaseOptions:   定义参数使用

    Examples
    ---------------------------------------------------------------------------------------------------
    # 获取所有参数
    >>> from options import BaseOptions
    >>> args = BaseOptions().parse()
    >>> # 取参数
    >>> args.参数名
    >>> # 新增参数
    >>> args.新参数名=参数值
    >>> # 修改参数
    >>> args.已有参数=修改的参数值
    >>>
```

```python
    """
    def __init__(self):
        """__init__:    初始化类,该类尚未初始化
        """
        self.initialized = False

    def initialize(self,parser):
        """initialize:   定义通用的参数
        Parameters:
        ------------------------------------------------------------------------------------------------------------
        >>> parser.add_argument('--dataroot'            # 变量名
                                , type=str               # 读取参数的类型
                                , required=False         # 可选项是否必须有
                                , default='a'            # 默认值,类型与type统一
                                , choices=['a','b']      # 参数值只能从几个选项里面选择
                                , help='11111'           # 该参数说明
                                )
        ------------------------------------------------------------------------------------------------------------
        """
        parser.add_argument('--dataroot'       # 变量名
                            , type=str               # 读取参数的类型
                            , required=False         # 可选项是否必须有
                            , default='a'            # 默认值,类型与type统一
                            , choices=['a','b']      # 参数值只能从几个选项里面选择
                            , help='11111'           # 该参数说明
                           )
        self.initialized = True
        return parser

    def gather_options(self):
        """gather_options:   初始化通用的参数(只初始化一次)
        """
        if not self.initialized:
            parser = argparse.ArgumentParser(formatter_class=argparse.ArgumentDefaultsHelpFormatter)
            parser = self.initialize(parser)
        return parser.parse_args()

    def parse(self):
        """parse:   解析参数
        """
        opt = self.gather_options()
```

```
60            self.opt = opt
61            return self.opt
62
```

3 三、 time 和 datetime 模块

3.1 time模块

- 用于满足简单的时间处理，如获取当前时间戳、日期、时间、休眠等。

方法	描述
time.ctime(seconds)	返回当前时间时间戳
time.localtime([seconds])	当前时间，以stuct_time时间类型返回
time.mktime(tuple)	将一个stuct_time时间类型转换成时间戳
time.strftime(format[, tuple])	将元组时间转换成指定格式。[tuple]不指定默认是当前时间
time.time0	返回当前时间时间戳
time.sleep(seconds)	延迟执行给定的秒数

```
>>> import time
>>> time.time()   # 1970-1-1 0:00:00 到当前时间的秒数
1625557742.433689
>>> time.ctime()   # UTC时间：世界标准时间
'Tue Jul  6 15:49:08 2021'
>>> time.localtime()  # 九元组
time.struct_time(tm_year=2021, tm_mon=7, tm_mday=6, tm_hour=15, tm_min=50, tm_sec=1, tm_wday=1, tm_yday=187, tm_isdst=0)
>>> t = time.localtime()
>>> t.tm_year
2021
>>> t.tm_mon
7
>>> t.tm_mday
6
>>> time.sleep(3)
>>> time.strftime('%Y-%m-%d %H:%M:%S')
'2021-07-06 15:52:43'
```

3.2 datetime模块

- 用来处理更复杂的日期和时间

类	描述
datetime.date	日期，年月日组成
datetime.datetime	包括日期和时间
datetime.time	时间，时分秒及微秒组成
datetime.timedelta	时间间隔
datetime.tzinfo	时区信息对象

```
>>> from datetime import datetime,date
>>> t1 = datetime.now()
>>> t1
datetime.datetime(2021, 7, 6, 16, 4, 54, 780362)
>>> t1.year, t1.month, t1.day, t1.hour, t1.minute, t1.second, t1.microsecond
(2021, 7, 6, 16, 4, 54, 780362)

"""将datetime对象转换成时间字符串"""
>>> datetime.strftime(t1, '%Y-%m-%d %H:%M:%S')
'2021-07-06 16:04:54'

"""将时间字符串转换成datetime对象"""
>>> datetime.strptime('2021-07-06 16:04:54', '%Y-%m-%d %H:%M:%S')
datetime.datetime(2021, 7, 6, 16, 4, 54)
>>> date.today()
datetime.date(2021, 7, 6)

# 打印年月日
>>> print(date.today())
2021-07-06

```

- 利用 `timedelta` 计算时间差额，100天20小时30分前、后是什么时间

```
>>> from datetime import datetime, timedelta

>>> time_delta = timedelta(days=100,hours=20,minutes=30)
>>> time_delta
datetime.timedelta(days=100, seconds=73800)

>>> time_now = datetime.now()
>>> time_now
datetime.datetime(2021, 7, 6, 16, 17, 59, 867774)
"""100天20小时30分前"""
>>> time_now - time_delta
datetime.datetime(2021, 3, 27, 19, 47, 59, 867774)
"""100天20小时30分后"""
```

```
>>> time_now + time_delta
datetime.datetime(2021, 10, 15, 12, 47, 59, 867774)

```

4 四、 sys 模块

```
import sys

"""platfrom方法识别操作系统"""
>>> print(sys.platform)
darwin

"""argv方法获取命令行参数"""
>>> print(sys.argv)
['/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package/11、Package_sys.py']

"""path 方法获取模块路径"""
>>> for path in sys.path:
>>>     print(path)
/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package
/Users/mengzhichao/Desktop/文件/16、python代码
/Users/mengzhichao/anaconda3/lib/python37.zip
。。。。。。

"""modules 方法查找已经导入的模块"""
>>> print(sys.modules.keys())
dict_keys(['sys', 'builtins', '_frozen_importlib', '等等'])

"""exit 方法退出程序"""
>>> for i in range(5):
>>>     print(i)
>>>     if i > 2:
>>>         sys.exit(2)
0
1
2
3

Process finished with exit code 2
```

5 五、 面向对象 回顾

5.1 对面向对象的理解

5.1.1 面向对象的编程（object oriented programming）

- 简称：OOP，是一种编程的思想。OOP把对象当成一个程序的基本单元，一个对象包含了数据和操作数据的函数。面向对象的出现极大的提高了编程的效率，使其编程的重用性增高。

5.1.2 python面向对象的术语

- 以下为基本常用术语
    1. 封装 （encapsulation）：把需要重用的函数或者功能封装，方便其他程序直接调用；
    2. 继承 （inheritance）子项继承父项的某些功能，在程序中表现某种联系；
    3. 多态 （polymorphism）：一个函数有多种表现形式，调用一个方法有多种形式，但是表现出的方法是不一样的；
    4. 类：对具有相同数据或者方法的一组对象的集合；
    5. 对象：对象是一个类的具体实例；
    6. 实例化：是一个对象实例话的实现；
    7. 标识：每个对象的实例都需要一个可以唯一标识这个实例的标记；
    8. 实例属性：一个对象就是一组属性的集合；
    9. 实例方法：所有存取或者更新对象某个实例一条或者多条属性函数的集合；
    10. 类属性：属于一个类中所有对象的属性；
    11. 类方法：那些无须特定的对性实例就能够工作的从属于类的函数。

5.1.3 函数编程和面向对象编程的区别

- 相同点：都是把程序进行封装、方便重复利用，提高效率。
- 不同点：函数重点是用于整体调用，一般用于一段不可更改的程序。仅仅是解决代码重用性的问题。

  而面向对象除了代码重用性。还包括继承、多态等。使用上更加灵活。

5.2 封装、继承、多态

5.2.1 封装（encapsulation）

- 封装：顾名思义就是将内容封装到某个地方，以后再去调用被封装在某处的内容。

对于面向对象的封装来说，其实就是使用构造方法将内容封装到 对象 中，然后通过对象直接或者 `self` 间接获取被封装的内容。

以下即简单的封装了吃、喝、睡三个技能。

```
class Foo:

    def __init__(self, name, age, gender):
        self.name = name
        self.age = age
        self.gender = gender

    def eat(self):
        print("%s,%s岁,%s,吃奶" % (self.name, self.age, self.gender))

    def drink(self):
        print("%s,%s岁,%s,喝水" % (self.name, self.age, self.gender))

    def sleep(self):
        print("%s,%s岁,%s,睡觉" % (self.name, self.age, self.gender))

```

5.2.2 继承（Inheritance）

- 继承：面向对象中的继承和现实生活中的继承相同，即：子可以继承父的内容。

  例如：

  1. 猫可以：喵喵叫、吃、喝、拉、撒
  2. 狗可以：汪汪叫、吃、喝、拉、撒

  公共的部分就是 吃、喝、拉、撒

```
class Animal:

    def eat(self):
        print("%s 吃 " % self.name)

    def drink(self):
        print("%s 喝 " % self.name)

    def shit(self):
        print("%s 拉 " % self.name)

    def pee(self):
        print("%s 撒 " % self.name)


class Cat(Animal):

```

```python
    def __init__(self, name):
        self.name = name
        self.breed = '猫'

    def cry(self):
        print('喵喵叫')


class Dog(Animal):

    def __init__(self, name):
        self.name = name
        self.breed = '狗'

    def cry(self):
        print('汪汪叫')
```

注意： 关于 多继承

1. 在Python中，如果父类和子类都重新定义了构造方法init( )，在进行子类实例化的时候，子类的构造方法不会自动调用父类的构造方法，必须在子类中显示调用；
2. Python的类可以继承多个类，Java和C#中则只能继承一个类；
3. Python的类如果继承了多个类，那么其寻找方法的方式有两种，分别是：深度优先和广度优先；
4. 当类是经典类时，多继承情况下，会按照深度优先方式查找，当类是新式类时，多继承情况下，会按照广度优先方式查找。

- 经典类 和 新式类 ，从字面上可以看出一个老一个新，新的必然包含了跟多的功能，也是之后推荐的写法，从写法上区分的话，如果 当前类或者父类继承了object类，那么该类便是新式类，否则便是经典类。
  即：
  1. 经典类： 未继承 `object` 类
  2. 新式类：继承 `object` 类

5.2.3 多态（Polymorphism）

- 首先Python不支持多态，也不用支持多态，python是一种多态语言，崇尚鸭子类型。

  在程序设计中，鸭子类型（英语：duck typing）是动态类型的一种风格。在这种风格中，一个对象有效的语义，不是由继承自特定的类或实现特定的接口，而是由当前方法和属性的集合决定。这个概念的名字来源于由James Whitcomb Riley提出的鸭子测试，“鸭子测试”可以这样表述：

> “当看到一只鸟走起来像鸭子、游泳起来像鸭子、叫起来也像鸭子，那么这只鸟就可以被称为鸭子。

在鸭子类型中，关注的不是对象的类型本身，而是它是如何使用的。例如，在不使用鸭子类型的语言中，我们可以编写一个函数，它接受一个类型为鸭的对象，并调用它的走和叫方法。在使用鸭子类型的语言中，这样的一个函数可以接受一个任意类型的对象，并调用它的走和叫方法。如果这些需要被调用的方法不存在，那么将引发一个运行时错误。任何拥有这样的正确的走和叫方法的对象都可被函数接受的这种行为引出了以上表述，这种决定类型的方式因此得名。

鸭子类型通常得益于不测试方法和函数中参数的类型，而是依赖文档、清晰的代码和测试来确保正确使用。从静态类型语言转向动态类型语言的用户通常试图添加一些静态的（在运行之前的）类型检查，从而影响了鸭子类型的益处和可伸缩性，并约束了语言的动态特性。

```
class A:
    def prt(self):
        print("A")

class B(A):
    def prt(self):
        print("B")

class C(A):
    def prt(self):
        print("C")

class D(A):
    pass

class E:
    def prt(self):
        print("E")

class F:
    pass

def test(arg):
    arg.prt()

a = A()
b = B()
c = C()
d = D()
e = E()
f = F()

test(a)
test(b)
test(c)
test(d)
test(e)
test(f)
```

- 没有谁规定test方法是接收的参数是什么类型的。test方法只规定，接收一个参数，调用这个参数的prt方法。在运行的时候如果这个参数有prt方法，python就执行，如果没有，python就报错，因为abcde都有prt方法，而f没有，所以得到了上边得结果，这就是python的运行方式。

5.3 面向对象的各种方法

5.3.1 静态方法(装饰器：@staticmethod)

- 意思是把 `@staticmethod` 下面的函数和所属的类截断了，这个函数就不属于这个类了，没有类的属性了，只不过还是要通过类名的方式调用。
- 错误的例子

```
"""错误示例"""
class Person(object):
    def __init__(self, name):
        self.name = name

    @staticmethod  # 把eat方法变为静态方法
    def eat(self):
        print("%s is eating" % self.name)

d = Person("xiaoming")
d.eat()

结果:
TypeError: eat() missing 1 required positional argument: 'self'
```

因为用静态方法把 `eat` 这个方法与 `Person` 这个类截断了，`eat` 方法就没有了类的属性了，所以获取不到 `self.name` 这个变量。

- 正确的例子

```
"""正确示例"""
class Person(object):
    def __init__(self, name):
        self.name = name

    @staticmethod  # 把eat方法变为静态方法
    def eat(x):
        print("%s is eating" % x)

d = Person("xiaoming")
d.eat("jack")

# 就把eat方法当作一个独立的函数给他传参就行了
```

5.3.2 类方法（装饰器：@classmethod）

- 类方法只能访问类变量，不能访问实例变量
- 错误的例子

```
'''错误示例'''

class Person(object):
    def __init__(self, name):
        self.name = name

    @classmethod  # 把eat方法变为类方法
    def eat(self):
        print("%s is eating" % self.name)


d = Person("xiaoming")
d.eat()
```

因为 `self.name` 这个变量是实例化这个类传进去的，类方法是不能访问实例变量的，只能访问类里面定义的变量

- 正确的例子

```
"""正确方法"""
class Person(object):
    name = 'mengzhichao'

    def __init__(self, name):
        self.name = name

    @classmethod  # 把eat方法变为类方法
    def eat(self):
        print("%s is eating" % self.name)


d = Person("xiaoming")
d.eat()


结果:
mengzhichao is eating
```

5.3.3 属性方法(@property)

- 把一个方法变成一个静态属性，属性就不用加小括号那样的去调用了
- 错误的例子

```
'''错误示例: '''

class Person(object):

    def __init__(self, name):
```

```
        self.name = name

    @property  # 把eat方法变为属性方法
    def eat(self):
        print("%s is eating" % self.name)


d = Person("xiaoming")
d.eat()

结果:
TypeError: 'NoneType' object is not callable

```

因为eat此时已经变成一个属性了， 不是方法了， 想调用已经不需要加()号了，直接d.eat就可以了

- 正确的例子

```
'''正确示例: '''

class Person(object):

    def __init__(self, name):
        self.name = name

    @property  # 把eat方法变为属性方法
    def eat(self):
        print("%s is eating" % self.name)


d = Person("xiaoming")
d.eat
```

5.4 高级面向对象

5.4.1 成员修饰符

- python的类中只有私有成员和公有成员两种，不像c++中的类有公有成员（public），私有成员(private)和保护成员(protected).并且python中没有关键字去修饰成员，默认python中所有的成员都是公有成员，但是私有成员是以两个下划线开头的名字标示私有成员， 私有成员不允许直接访问 ，只能通过内部方法去访问， 私有成员也不允许被继承 。

```
class a:  # 说明父类的私有成员无法在子类中继承
    def __init__(self):
        self.ge=123
        self.__gene=456

class b(a):
```

```
    def __init__(self,name):
        self.name=name
        self.__age=18
        super(b,self).__init__()  # 这一行会报错
    def show(self):
        print(self.name)
        print(self.__age)
        print(self.ge)
        print(self.__gene)  # 这一行也会报错
obj=b("xiaoming")
print(obj.name)
print(obj.ge)
# print(obj.__gene)  # 这个也会报错
obj.show()
```

5.4.2 特殊成员

5.4.2.1 __init__

- 可以简单的理解为类的 `构造` 方法（实际并不是构造方法，只是在类生成对象之后就会被执行）

5.4.2.2 __del__

- `析构` 方法，当对象消亡的时候（被解释器的垃圾回收的时候会执行这个方法）这个方法默认是不需要写的，不写的时候，默认是不做任何操作的。因为你不知道对象是在什么时候被垃圾回收掉，所以，除非你确实要在这里面做某些操作，不然不要自定义这个方法。

5.4.2.3 __call__

- 方法在类的对象被执行的时候（obj()或者 类()()）会执行。

5.4.2.4 __int__

- 在对象被int()包裹的时候会被执行，例如int(obj)如果obj对象没有、**int**方法，那么就会报错。在这个方法中返回的值被传递到int类型中进行转换。

5.4.2.5 __str__

- 当对象被str(obj)包裹的时候，如果对象中没有这个方法将会报错，如果有这个方法，str()将接收这个方法返回的值在转换成字符串。

5.4.2.6 __add__

- 在两个对象相加的时候，调用第一个对象的**add**方法，将第二个对象传递进来，至于怎么处理以及返回值，那是程序员自定义的，就如下面的例子：

```
class abc:
    def __init__(self,age):
        self.age=age
    def __add__(self,obj):
        return self.age-obj.age  # 这里将加改成减，方便理解
a1=abc(18)
a2=abc(20)
print(a1+a2)
#执行结果：-2
```

5.4.2.7 __dict__

- __dict__方法在类里面有，在对象里面也有，这个方法是以字典的形式列出类或对象中的所有成员。就像下面的例子：

```
class abc:
    def __init__(self, age, gender):
        self.age = age
        self.gender = gender

    def __add__(self, obj):
        return self.age + obj.age


a1 = abc(18, 'male')
print(abc.__dict__)
print(a1.__dict__)
```

5.4.2.8 _getitem__ , _ setitem, delitem__

- **getitem**方法匹配 对象[索引] 这种方式， **setitem**匹配 对象[索引]=value 这种方式， **delitem** 匹配 del 对象[索引] 这种方式，例子如下:

```
class Foo:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __getitem__(self, item):  # 匹配:对象[item]这种形式
        return item + 10

    def __setitem__(self, key, value):  # 匹配:对象[key]=value这种形式
        print(key, value)

    def __delitem__(self, key):  # 匹配:del 对象[key]这种形式
        print(key)


li = Foo("alex", 18)
```

```
print(li[10])
li[10] = 100
del li[10]

```

5.4.2.9 __iter__

- 类的对象如果想要变成一个可迭代对象，那么对象中必须要有**iter**方法，并且这个方法返回的是一个迭代器。

  for 循环的对象如果是一个可迭代的对象，那么会先执行对象中的**iter**方法，获取到迭代器，然后再执行迭代器中的**next**方法获取数据。如果for循环的是一个迭代器，那么直接执行迭代器中的**next**方法。

```
class Foo:
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def __iter__(self):
        return iter([1,2,3,4,5])  # 返回的是一个迭代器
li=Foo("alex",18)

# 1.如果类中有__iter__方法，他的对象就是可迭代对象
# 2.对象.__iter()的返回值是一个迭代器
# 3.for循环的如果是迭代器，直接执行.next方法
# 4.for循环的如果是可迭代对象，先执行对象.__iter()，获取迭代器再执行next

for i in li:
    print(i)
#执行结果：
1
2
3
4
5
```

5.4.2.10 isinstance和issubclass

- isinstance可以判断一个变量是否是某一种数据类型，其实，isinstance不只可以判断数据类型，也可以判断对象是否是这个类的对象或者是这个类的子类的对象，代码如下：

```
class Foo:
    def __init__(self, name, age):
        self.name = name
        self.age = age


class Son(Foo):
    pass


obj = Son("xiaoming", 18)
```

```
print(isinstance(obj, Foo))

执行结果：True

```

issubclass用来判断一个类是否是某个类的子类，返回的是一个bool类型数据，代码如下：

```
class Foo:
    def __init__(self, name, age):
        self.name = name
        self.age = age


class Son(Foo):
    pass


obj = Son("xiaoming", 18)
print(issubclass(Son, Foo))
执行结果：True
```

5.4.3 类与对象

- `\__new____metaclass__`

  在 `python` 中，一切皆对象，我们定义的类其实。。。也是一个对象，那么，类本身是谁的对象呢？在python2.2之前（或者叫经典类中），所有的类，都是class的对象，但是在新式类中，为了将类型（int,str,float等）和类统一，所以，所有的类都是type类型的对象。当然，这个规则可以被修改，在类中有一个属性 **metaclass** 可以指定当前类该由哪个类进行实例化。而创建对象过程中，其实构造器不是**init**方法，而是**new**方法，这个方法会返回一个对象，这才是对象的构造器。下面是一个解释类实例化对象内部实现过程的代码段：

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 14、类与对象.py
@time: 2021-07-13 10:01
**************************************************
"""


class Mytype(type):
    def __init__(self, what, bases=None, dict=None):
        super(Mytype, self).__init__(what, bases, dict)

    def __call__(self, *args, **kwargs):
        obj = self.__new__(self)
        self.__init__(obj, *args, **kwargs)
```

```python
        return obj


class Foo:
    __metaclass__ = Mytype

    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __new__(cls, *args, **kwargs):
        return object.__new__(cls)


obj = Foo("xiaoming", 18)
print(obj.name, obj.age)

```

5.4.4 异常处理

5.4.4.1 异常处理示例

- `python` 中使用 `try except finally` 组合来实现异常扑捉，不像 `java` 中是使用 `try catch finally……` 其中， `except` 中的 `Exception` 是所有异常的父类，下面是一个异常处理的示例：

```python
try:
    # int("aaa")  # 可能出现异常的代码
    # list(range(10))[11]
    None > 0
except IndexError as e:  # 捕捉索引异常的子异常，注意，这里的as e在老版本的py中可以写成，e但是新版本中用as e,",e"未来可能会淘汰
    print("IndexError:", e)
except ValueError as e:  # 捕捉value错误的子异常
    print("ValueError:", e)
except Exception as e:  # 如果上面两个异常没有捕获到，那么使用Exception捕获，Exception能够捕获所有的异常
    print("Exception:", e)
else:  # 如果没有异常发生，执行else中的代码块
    print("true")
finally:  # 不管是否发生异常，在最后都会执行finally中的代码，假如try里面的代码正常执行，先执行else中的代码，再执行finally中的代码
    print("finally")

Exception: '>' not supported between instances of 'NoneType' and 'int'
finally
```

5.4.4.2 自定义异常处理

- 那么既然 `Exception` 是所有异常的父类，我们可以自已定义 `Exception` 的子类，实现自定

义异常处理，下面就是实现例子：

```
class OldBoyError(Exception):  # 自定义错误类型
    def __init__(self, message):
        self.message = message

    def __str__(self):  # 打印异常的时候会调用对象里面的__str__方法返回一个字符串
        return self.message

try:
    raise OldBoyError("我错了...")  # raise是主动抛出异常，可以调用自定义的异常抛出异常
except OldBoyError as e:
    print(e)

我错了...
```

异常处理里面还有一个断言，一般用在判断执行环境上面，只要断言后面的条件不满足，那么就抛出异常，并且后面的代码不执行了。

```
print(123)
assert 1 == 2  # 断言，故意抛出异常，做环境监测用，环境监测不通过，报错并结束程序
print("456")

结果：
assert 1 == 2  # 断言，故意抛出异常，做环境监测用，环境监测不通过，报错并结束程序
AssertionError
```

5.4.5 反射、自省

- python中的反射/自省的实现，是通过 `hasattr` 、 `getattr` 、 `setattr` 、 `delattr` 四个内置函数实现的，其实这四个内置函数不只可以用在类和对象中，也可以用在模块等其他地方，只是在类和对象中用的很多，所以单独提出来进行解释。
    1. `hasattr(key)` 返回的是一个 `bool` 值，判断某个成员或者属性在不在类或者对象中;
    2. `getattr(key,default=xxx)` 获取类或者对象的成员或属性，如果不存在，则会抛出 `AttributeError` 异常,如果定义了default那么当没有属性的时候会返回默认值;
    3. `setattr(key,value)` 假如有这个属性，那么更新这个属性，如果没有就添加这个属性并赋值 `value` ;
    4. `delattr(key)` 删除某个属性。

注意，上面的key都是字符串，而不是变量，也就是说可以通过字符串处理类中的成员或者对象中的属性。下面是一个例子代码：

```
class Foo:
```

```
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def show(self):
        return self.name, self.age


obj = Foo("xiaoming", 18)
print(getattr(obj, "name"))

setattr(obj, "k1", "v1")  # 更新、或添加并赋值
print(obj.k1)

print(hasattr(obj, "k1"))
delattr(obj, "k1")  # 删除

show_fun = getattr(obj, "show")
print(show_fun())


结果:
xiaoming
v1
True
('xiaoming', 18)
```

- 反射/自省能够直接访问以及修改运行中的类和对象的成员和属性，这是一个很强大的功能，并且并不像java中效率很低，所以用的很多。

  下面是一个反射/自省用在模块级别的例子：

```
import s2
operation=input("请输入URL:")
if operation in s2.__dict__:
    getattr(s2,operation)()
else:
    print("404")

#模块s2中的代码:
def f1():
    print("首页")
def f2():
    print("新闻")
def f3():
    print("精选")

执行结果:
请输入URL:f1
首页
```

5.4.6 单例模式

- 简单的介绍一个简单的设计模式：单例模式。在面向对象中的单例模式就是一个类只有一个对象，所有的操作都通过这个对象来完成，这就是面向对象中的单例模式，下面是实现代码

```
class Foo:  # 单例模式
    __v = None

    @classmethod
    def ge_instance(cls):
        if cls.__v:
            return cls.__v
        else:
            cls.__v = Foo()
            return cls.__v


obj1 = Foo.ge_instance()
print(obj1)
obj2 = Foo.ge_instance()
print(obj2)
obj3 = Foo.ge_instance()
print(obj3)


结果:
<__main__.Foo object at 0x106334780>
<__main__.Foo object at 0x106334780>
<__main__.Foo object at 0x106334780>
```

6 六、 `json` 模块

6.1 序列化概念

1. 我们平常使用的python对象所进行的操作是在内存中，当程序关闭就会被清空，所以我们需要用一种合适的方法将这些数据保存下来；
2. 为了将我们的数据进行永久存储，需要引入序列化（pickling/serialization）的概念；
3. 序列化的定义：将复杂的python数据结果转换成一个二进制数据集合（数据流）；
4. 反序列化：从数据流（字符串形式）重新构造复杂的python数据结；
5. 序列化的好处：我们可以通过网络或本地存储介质讲这些数据流保存或传输；
6. 序列化的方法：引入 `json` 、 `pickling` 、 `marshal` 、 `shelve` ，其中最常用的是 `json` 。

6.2 json标准库

6.2.1 基本用法

1. `json` 模块的序列化和反序列化方法分别是 `dumps` 和 `loads` ， `json.dumps()` 和 `json.dump()` 将一个python对象转换成 `json` 串， `json.loads()` 和 `json.load()` 将一个 `json` 串还原成 `python` 对象。
2. `json.dump()` 和 `json.dumps()` 的区别
   `json.dump()` 处理的是文件对象，而 `json.dumps()` 处理的是字符串对象。
3. `json.load()` 和 `json.loads()` 的区别
   `json.load()` 处理的是文件对象，而 `json.loads()` 处理的是字符串对象。

6.2.2 json.dumps()

- 将python的数据类型转换成json字串

  语法格式：

```
json.dumps(obj
        , skipkeys=False# 是否允许JSON字串编码字典对象时，字典的key不是字符串类型（默认是不允许
        , ensure_ascii=True
        , check_circular=True
        , allow_nan=True
        , cls=None
        , indent=None # 定义缩进大小
        , separators=None # 是一个元组，定义分隔符类型
        , encoding="utf-8"
        , default=None
        , sort_keys=False # 是否排序
        , **kw)
```

```
>>> import json
>>> json.dumps([])
Out[3]: '[]'
>>> json.dumps('String')
Out[4]: '"String"'
>>> json.dumps(1)
Out[5]: '1'
>>> json.dumps({"name":"tyson","age":23})
Out[6]: '{"name": "tyson", "age": 23}'
```

使用参数能让JSON字串格式化输出：

```
>>> print(json.dumps({'a': 'Runoob', 'b': 7}, sort_keys=True, indent=4, separators=(',', ': ')))
{
    "a": "Runoob",
    "b": 7
}

```

- 较重要的 `参数` ：
  `sort_keys` ： 是否排序
  `indent` ： 定义缩进大小
  `separators` ： 是一个元组，定义分隔符的类型
  `skipkeys` :是否允许 `JSON` 字串编码字典对象时，字典的key不是字符串类型（默认是不允许）

```
a = ['foo', {'bar': ('baz', None, 1.0, 2)}]
s = json.dumps(a
               , sort_keys=True
               , indent=4
               , separators=('!', '?')
              ) #分隔符这里只是测试，一般保持默认逗号和分号即可
print(s)

```

```

[
    "foo"!
    {
        "bar"?[
            "baz"!
            null!
            1.0!
            2
        ]
    }
]
```

```python
import json

try:
    data = {'a': 1, 'b': 2, (1,): 3}
    json.dumps(data)
except TypeError as e:
    print(e)  # keys must be a string

print('='*25)
print(json.dumps(data, skipkeys=True))  # {"a": 1, "b": 2}

print('='*25)
data = {'a': 1, 'b': 2, 1: 3}
print(json.dumps(data))

result:
keys must be a string
{"a": 1, "b": 2}
{"a": 1, "1": 3, "b": 2}

```

- Python 原始类型向json类型的转换对照表

python	json
dict	object
list,tuple	array
str,unicode	string
int,long,float	number
True	true
False	false
None	null

6.2.3 json.loads()

- 将json字串转换成python的数据类型。也就是 反序列化 。

  语法格式:

```
loads(s
 , *
 , encoding=None
 , cls=None
 , object_hook=None
 , parse_float=None
 , parse_int=None
 , parse_constant=None
 , object_pairs_hook=None
 , **kw):
```

```
json.loads('{"a":"Runoob","b":7}')
Out[12]: {'a': 'Runoob', 'b': 7}
```

- json字串转换成python对象类型对照表

JSON	Python
object	dict
array	list
string	unicode
number(int)	int
number(real)	float
true	True
false	False
null	

6.2.4 json.dump()

- 将序列化之后形成的JSON字串保存到文件。
  语法格式:

```
dump(obj
, fp
, *
, skipkeys=False
, ensure_ascii=True
, check_circular=True
, allow_nan=True
, cls=None
, indent=None
, separators=None
, default=None
, sort_keys=False
, **kw):
```

```python
import json

a = {"name": "tyson", "age": 21, "sex": "boy"}

# 第一种方法将JSON字串写入文件中
# 该方法不需要close
with open("./save/json.txt", "a") as f:
    f.write(json.dumps(a, indent=4))

# 第二种方法将JSON字串写入文件中
f = open("./save/json.txt", "a")
json.dump(a, f, indent=4)
f.close()

"""json.txt"""
{
    "name": "tyson",
    "age": 21,
    "sex": "boy"
}{
    "name": "tyson",
    "age": 21,
    "sex": "boy"
}

```

6.2.5 json.load()

- 从文件接受JSON字串，并反序列化成为Python的数据类型。
  语法格式:

```
load(fp
, *
, cls=None
, object_hook=None
, parse_float=None
, parse_int=None
, parse_constant=None
, object_pairs_hook=None
, **kw):
```

```
import json

a = {"name": "tyson", "age": 21, "sex": "boy"}

# 第一种方法将JSON字串写入文件中
# 该方法不需要close
with open("./save/json", "a") as f:
    f.write(json.dumps(a))

# 第一种方法将文件中的JSON字串转反序列化成Python的数据类型
with open("./save/json", "r") as f:
    print(json.loads(f.read()))

# json
{"age": 21, "name": "tyson", "sex": "boy"}


result:
{'name': 'tyson', 'age': 21, 'sex': 'boy'}

```

6.3 类的序列化

- 类的序列化

```

"""类的序列化"""
import json


class student(object):
    def __init__(self, name, sex, age):
        self.name = name
        self.sex = sex
        self.age = age


```

```
tyson = student("tyson", "boy", 21)
print(json.dumps(tyson, default=lambda obj: obj.__dict__))


# python的JSON想办法能将各种数据对象都转换为JSON，其中可选参数default就是用来提供给用户自行定义转换函数的
# 所以上面的方法与下面这个是等同的
def stuclass2json(classObject):
    return {
        'name': classObject.name,
        'sex': classObject.sex,
        'age': classObject.age
    }


print(json.dumps(tyson, default=stuclass2json))

result:
{"name": "tyson", "sex": "boy", "age": 21}
{"name": "tyson", "sex": "boy", "age": 21}
```

6.4 无法序列化问题

- json.dump()或者json.dumps()无法序列化，报xxx is not JSON serializable的解决方案

```
import json
import numpy as np

params = {'1':2,'3':np.arange(10)}

json.dump(params,open('./test.txt',mode='w+'),indent=2,ensure_ascii=False)

"""报错"""
TypeError: Object of type ndarray is not JSON serializable
```

- 创建一个类来继承json.JSONEncoder， **重写** json.JSONEncoder中的方法，即自定义序列化方法

```
class NpEncoder(json.JSONEncoder):
    def default(self, obj):
        if isinstance(obj, np.integer):
            return int(obj)
        elif isinstance(obj, np.floating):
            return float(obj)
        elif isinstance(obj, np.ndarray):
            return obj.tolist()
        else:
            return super(NpEncoder, self).default(obj)

```

```python
json.dump(params
          ,open('./test.txt',mode='w+')
          ,indent=2
          ,cls =NpEncoder
          ,ensure_ascii=False
         )
```

```
{
  "1": 2,
  "3": [
    0,
    1,
    2,
    3,
    4,
    5,
    6,
    7,
    8,
    9
  ]
}
```

7 七、 pickle 模块

7.1 pickle模块简介

7.1.1 什么是序列化

- 在程序运行的过程中，所有的变量都是在内存中，比如，定义一个 dict：

```
a = {'name':'Bob','age':20,'score':90}
```

  字典 a 可以随时修改变量，比如把 `name` 改成 `'Bill'` ，但是一旦程序结束，变量所占用的内存就被操作系统全部回收。如果没有把修改后的 `'Bill'` 存储到磁盘上，下次重新运行程序，变量又被初始化为 `'Bob'` 。

  我们把变量从内存中变成可存储或传输的过程称之为序列化， 在 `Python` 中叫 `pickling` 。序列化之后，就可以把序列化后的内容写入磁盘，或者通过网络传输到别的机器上。反过来，把变量内容从序列化的对象重新读到内存里称之为反序列化，即 `unpickling` 。

7.1.2 pickle特点

- pickle 序列化的程序有什么特点：
  1. pickle 序列化后的数据，可读性差，人一般无法识别；
  2. pickle 模块只能在 python 中使用，且只支持同版本，不能跨平台使用；
  3. Python 中所有的数据类型（列表，字典，集合，类等）都可以用 pickle 来序列化。
  4. 需导入 pickle 模块 — import pickle

7.2 pickle模块常用方法

- `pickle` 常用方法有：
  1. `dumps` ： 把 obj 对象序列化后以 bytes 对象返回，不写入文件；
  2. `loads` ： 从 bytes 对象中读取一个反序列化对象，并返回其重组后的对象；
  3. `dump` ： 序列化对象，并将结果数据流写入到文件对象中；
  4. `load` ： 反序列化对象，将文件中的数据解析为一个Python对象。

7.2.1 pickle.dumps(obj)

- 把 obj 对象序列化后以 bytes 对象返回，不写入文件

  下面代码分别对列表l1、元组t1、字典dic1进行序列化操作，打印后可以看到结果是一堆二进制乱码。代码如下所示：

```python
import pickle

l1 = [1, 2, 3, 4, 5]
t1 = (1, 2, 3, 4, 5)
dic1 = {"k1": "v1", "k2": "v2", "k3": "v3"}

res_l1 = pickle.dumps(l1)
res_t1 = pickle.dumps(t1)
res_dic = pickle.dumps(dic1)

print(res_l1)
print(res_t1)
print(res_dic)
```

运行结果如下：

```
b'\x80\x03]q\x00(K\x01K\x02K\x03K\x04K\x05e.'
b'\x80\x03(K\x01K\x02K\x03K\x04K\x05tq\x00.'
b'\x80\x03}q\x00(X\x02\x00\x00\x00k1q\x01X\x02\x00\x00\x00v1q\x02X\x02\x00\x00\x00k2q\x03X\x02\x00\x00\x00v2q\x04X\x02\x00\x00\x00k3q\x05X\x02\x00\x00\x00v3q\x06u.'
```

7.2.2 pickle.loads(bytes_object)

- 从 bytes 对象中读取一个反序列化对象，并返回其重组后的对象

  下面代码是先分别对列表l1、元组t1、字典dic1进行序列化，然后再对序列化后的对象进行反序列化操作，打印后可以看到序列化之前是什么类型的数据，反序列化后其数据类型不变。代码如下所示：

```python
import pickle

l1 = [1, 2, 3, 4, 5]
t1 = (1, 2, 3, 4, 5)
dic1 = {"k1": "v1", "k2": "v2", "k3": "v3"}

res_l1 = pickle.dumps(l1)
res_t1 = pickle.dumps(t1)
res_dic = pickle.dumps(dic1)

print(pickle.loads(res_l1), type(pickle.loads(res_l1)))
print(pickle.loads(res_t1), type(pickle.loads(res_t1)))
print(pickle.loads(res_dic), type(pickle.loads(res_dic)))
```

运行结果如下：

```
[1, 2, 3, 4, 5] <class 'list'>
(1, 2, 3, 4, 5) <class 'tuple'>
{'k1': 'v1', 'k2': 'v2', 'k3': 'v3'} <class 'dict'>
```

7.2.3 pickle.dump(obj , file)

- 序列化对象，并将结果数据流写入到文件对象中
  下面代码分别把 列表l1、元组t1、字典dic1 进行序列化操作，并保存到一个文件中以实现永久保存。代码入下所示：

```
import pickle

l1 = [1, 2, 3, 4, 5]
t1 = (1, 2, 3, 4, 5)
dic1 = {"k1": "v1", "k2": "v2", "k3": "v3"}
# 把列表l1序列化进一个文件f1中
with open("./f1.pkl", "wb") as f:
    pickle.dump(l1, f)
    pickle.dump(t1, f)
    pickle.dump(dic1, f)
```

  序列化后在当前目录下生成一个f1.pkl文件，打开文件f1可以看到一堆乱码，如下所示：

```
]q(K•K•K•K•K•e.(K•K•K•K•K•tq.}q(X•k1q•X•v1q•X•k2q•X•v2q•X•k3q•X•v3q•u.
```

7.2.4 pickle.load(file)

- 反序列化对象，将文件中的数据解析为一个Python对象
  下面代码是先对列表l2进行序列化并保存到文件f2.pkl中，然后再对文件f2进行反序列化，得到文件f2里保存的数据

```
import pickle

l2 = [1, 2, 3, 4, 5, 6]
# 把列表l2序列化进一个文件f2中
with open("f2.pkl", "wb") as f:
    pickle.dump(l2, f)
with open("f2.pkl", "rb") as f:
    res = pickle.load(f)
    print(res, type(res))
```

  运行结果：

```
"反序列化之后，打印数据及其类型可以看到："
[1, 2, 3, 4, 5, 6] <class 'list'>
```

8 八、 logging 模块

8.1 日志

8.1.1 作用

- 在部署项目时，不可能直接将所有的信息都输出到控制台中，我们可以将这些信息记录到日志文件中，这样不仅方便我们查看程序运行时的情况，也可以在项目出现故障时根据运行时产生的日志快速定位问题出现的位置。
- 日志是一种可以追踪某些软件运行时所发生事件的方法。软件开发人员可以向他们的代码中调用日志记录相关的方法来表明发生了某些事情。一个事件可以用一个可包含可选变量数据的消息来描述。此外，事件也有重要性的概念，这个重要性也可以被称为严重性级别（level）。

8.1.2 日志级别

- `Python` 标准库 `logging` 用作记录日志，默认分为六种日志级别（括号为级别对应的数值）。

日志等级（Level）	说明
`NOTSET`	没有设置，默认为WARNING等级 当在WARNING或WARNING之上等级的才记录日志信息
`DEBUG`	最详细的日志信息，典型应用场景是 问题诊断,程序调试bug
`INFO`	程序正常运行时使用,信息详细程度仅次于DEBUG，通常只记录关键节点信息，用于确认一切都是按照我们预期的那样进行工作
`WARNING`	当某些不期望的事情发生时记录的信息，但是此时应用程序还是正常运行的 如:用户登录密码错误、磁盘可用空间较低
`ERROR`	由于一个更严重的问题导致某些功能不能正常运行时记录的信息，如:IO操作失败
`CRITICAL`	当发生严重错误，导致应用程序不能继续运行时记录的信息 如:磁盘空间为空，一般很少使 用

- 日志等级从高到低顺序

> DEBUG < INFO < WARNING < ERROR < CRITICAL

- 日志的级别也可以自己定义。
- `logging` 执行时输出大于等于设置的日志级别的日志信息，如设置日志级别是 INFO，则

INFO、WARNING、ERROR、CRITICAL 级别的日志都会输出。

这样根据等级筛选可以避免我们在排查故障时**被淹没在日志的海洋里**。

8.1.3 日志的流程

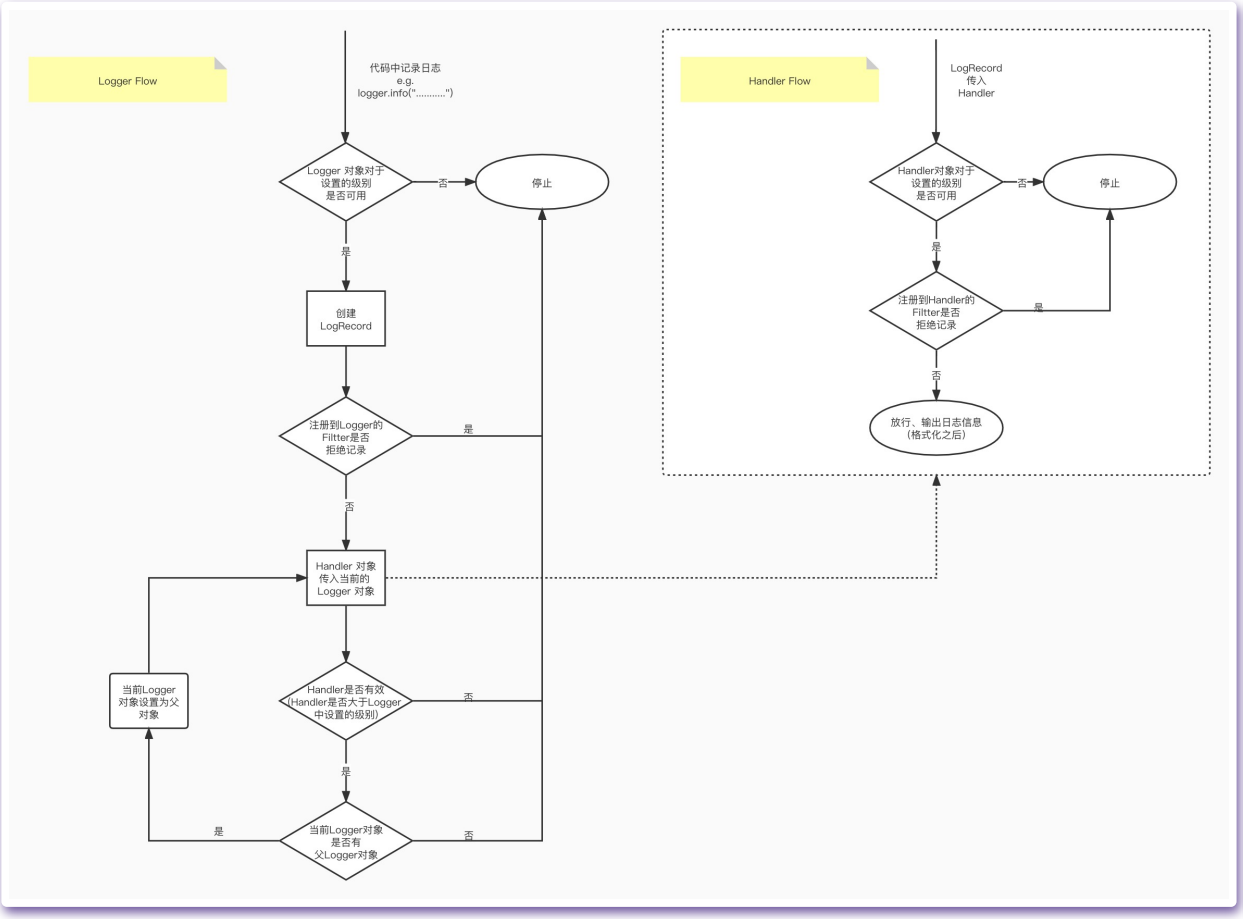
8.1.3.1 类别说明

- 类别说明

类别	说明
`Logger`	日志，暴露函数给应用程序，基于日志记录器和过滤器级别决定哪些日志有效。
`LogRecord`	日志记录器，将日志传到相应的处理器处理。
`Handler`	处理器, 将(日志记录器产生的)日志记录发送至合适的目的地。
`Filter`	过滤器, 提供了更好的粒度控制,它可以决定输出哪些日志记录。
`Formatter`	格式化器, 指明了最终输出中日志记录的布局。

8.1.3.2 流程说明

- 流程图



备注：

1. 判断 Logger 对象对于设置的级别是否可用，如果可用，则往下执行，否则，流程结束；
2. 创建 LogRecord 对象，如果注册到 Logger 对象中的 Filter 对象过滤后返回 False，则不记录日志，流程结束，否则，则向下执行；
3. LogRecord 对象将 Handler 对象传入当前的 Logger 对象，（图中的子流程）如果 Handler 对象的日志级别大于设置的日志级别，再判断注册到 Handler 对象中的 Filter 对象过滤后是否返回 True 而放行输出日志信息，否则不放行，流程结束；
4. 如果传入的 Handler 大于 Logger 中设置的级别，也即 Handler 有效，则往下执行，否则，流程结束；
5. 判断这个 Logger 对象是否还有父 Logger 对象，如果没有（代表当前 Logger 对象是最顶层的 Logger 对象 root Logger），流程结束。否则将 Logger 对象设置为它的父 Logger 对象，重复上面的 3、4 两步，输出父类 Logger 对象中的日志输出，直到是 root Logger 为止。

8.1.4 日志输出格式：

- 日志的输出格式可以认为设置，默认格式如下：

> WARNING : root : warn message
> 日志级别 ： Logger实例名称 ： 日志消息内容

- 一个事件通常需要包括以下几个内容：
    1. 事件发生时间
    2. 事件发生位置
    3. 事件的严重程度–日志级别
    4. 事件内容

  上面这些都是一条日志记录中可能包含的字段信息，当然还可以包括一些其他信息，如进程ID、进程名称、线程ID、线程名称等。

  输出一条日志时，日志内容和日志级别是需要开发人员明确指定的。对于而其它字段信息，只需要是否显示在日志中就可以了。

8.2 logging模块简介

8.2.1 logging模块的使用方式介绍

8.2.1.1 日志记录

- `logging` 模块提供了两种记录日志的方式：
    1. 使用 `logging` 提供的模块级别的函数

2. 使用`Logging`日志系统的四大组件

其实，`logging`所提供的模块级别的日志记录函数也是对`logging`日志系统相关类的封装而已。

- `logging`模块定义的模块级别的常用函数

函数	说明
logging.debug(msg, *args, **kwargs)	创建一条严重级别为DEBUG的日志记录
logging.info(msg, *args, **kwargs)	创建一条严重级别为INFO的日志记录
logging.warning(msg, *args, **kwargs)	创建一条严重级别为WARNING的日志记录
logging.error(msg, *args, **kwargs)	创建一条严重级别为ERROR的日志记录
logging.critical(msg, *args, **kwargs)	创建一条严重级别为CRITICAL的日志记录
logging.log(level, *args, **kwargs)	创建一条严重级别为level的日志记录
logging.basicConfig(**kwargs)	对root logger进行一次性配置

其中`logging.basicConfig(**kwargs)`函数用于指定“要记录的日志级别”、“日志格式”、“日志输出位置”、“日志文件的打开模式”等信息，其他几个都是用于记录各个级别日志的函数。

8.2.1.2 四大组件

- logging模块四大组件

组件	说明
`loggers`	提供应用程序代码直接使用的接口
`handlers`	用于将日志记录发送到指定的目的位置
`filters`	提供更细粒度的日志过滤功能，用于决定哪些日志记录将会被输出（其它的日志记录将会被忽略）
`formatters`	用于控制日志信息的最终输出格式

- 说明：

`logging`模块提供的模块级别的那些函数实际上也是通过这几个组件的相关实现类来记录日志的，只是在创建这些类的实例时设置了一些默认值。
`logging`模块就是通过这些组件来完成日志处理的，上面所使用的`logging`模块级别的函数也是通过这些组件对应的类来实现的。

- 这些组件的关系：

1. 日志器（`logger`）需要通过处理器（`handler`）将日志信息输出到目标位置，如：文件、sys.stdout、网络等；

2. 不同的处理器（`handler`）可以将日志输出到不同的位置；
3. 日志器（`logger`）可以设置多个处理器（`handler`）将同一条日志记录输出到不同的位置；
4. 每个处理器（`handler`）都可以设置自己的过滤器（`filter`）实现日志过滤，从而只保留感兴趣的日志；
5. 每个处理器（`handler`）都可以设置自己的格式器（`formatter`）实现同一条日志以不同的格式输出到不同的地方。

- 简单点说：

> 日志器（`logger`）是入口，真正干活儿的是处理器（`handler`），处理器（`handler`）还可以通过过滤器（`filter`）和格式器（`formatter`）对要输出的日志内容做过滤和格式化等处理操作。

8.3 logging模块基本使用（basicConfig）

8.3.1 basicConfig()方法

- `logging` 使用非常简单，使用 `basicConfig()` 方法就能满足基本的使用需要，如果方法没有传入参数，会根据默认的配置创建Logger 对象，默认的日志级别被设置为 **WARNING**，默认的日志输出格式如上图，该函数可选的参数如下表所示。

参数名称	参数描述
`filename`	日志输出到文件的文件名
`filemode`	文件模式，r[+]、w[+]、a[+]
`format`	日志输出的格式
`datefat`	日志附带日期时间的格式
`style`	格式占位符，默认为 “%” 和 “{}”
`level`	设置日志输出级别
`stream`	定义输出流，用来初始化 StreamHandler 对象，不能 filename 参数一起使用，否则会ValueError 异常
`handles`	定义处理器，用来创建 Handler 对象，不能和 filename 、stream 参数一起使用，否则也会抛出 ValueError 异常

- 示例

```python
import logging

logging.basicConfig()
logging.debug('This is a debug message')
logging.info('This is an info message')
logging.warning('This is a warning message')
logging.error('This is an error message')
logging.critical('This is a critical message')

"""输出"""
WARNING:root:This is a warning message
ERROR:root:This is an error message
CRITICAL:root:This is a critical message
```

日志器设置的日志格式默认是 `BASIC_FORMAT`

> “%(levelname)s:%(name)s:%(message)s”

- 日志记录输出到文件

  在logging模块提供的日志记录函数所使用的日志器设置的处理器所指定的日志输出位置默认为:

> sys.stderr

```python
import logging

logging.basicConfig(filename="test.log"
                    , filemode="w"
                    , format="%(asctime)s %(name)s:%(levelname)s:%(message)s"
                    , datefmt="%d-%m-%Y %H:%M:%S"
                    , level=logging.DEBUG
                    )

logging.debug('This is a debug message')
logging.info('This is an info message')
logging.warning('This is a warning message')
logging.error('This is an error message')
logging.critical('This is a critical message')

"""生成的日志文件 test.log , 内容如下"""
05-01-2022 10:26:19 root:DEBUG:This is a debug message
05-01-2022 10:26:19 root:INFO:This is an info message
05-01-2022 10:26:19 root:WARNING:This is a warning message
05-01-2022 10:26:19 root:ERROR:This is an error message
05-01-2022 10:26:19 root:CRITICAL:This is a critical message
```

8.3.2 捕获异常

- 但是当发生异常时，直接使用无参数的 debug()、info()、warning()、error()、critical() 方法并不能记录异常信息，需要设置 exc_info 参数为 True 才可以，或者使用 exception() 方法，还可以使用 log() 方法，但还要设置日志级别和 exc_info 参数。

```
import logging

logging.basicConfig(filename="test.log"
                    , filemode="w"
                    , format="%(asctime)s %(name)s:%(levelname)s:%(message)s",
                    datefmt="%d-%M-%Y %H:%M:%S"
                    , level=logging.DEBUG
                    )
a = 5
b = 0
try:
    c = a / b
except Exception as e:
    # 下面三种方式三选一，推荐使用第一种
    logging.exception("Exception occurred")
    logging.error("Exception occurred", exc_info=True)
    logging.log(level=logging.DEBUG, msg="Exception occurred", exc_info=True)

"""生成的日志文件 test.log ，内容如下"""
05-30-2022 10:30:10 root:ERROR:Exception occurred
Traceback (most recent call last):
  File "/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package/13、logging.py", line 54, in <module>
    c = a / b
ZeroDivisionError: division by zero
05-30-2022 10:30:10 root:ERROR:Exception occurred
Traceback (most recent call last):
  File "/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package/13、logging.py", line 54, in <module>
    c = a / b
ZeroDivisionError: division by zero
05-30-2022 10:30:10 root:DEBUG:Exception occurred
Traceback (most recent call last):
  File "/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package/13、logging.py", line 54, in <module>
    c = a / b
ZeroDivisionError: division by zero
```

8.3.3 格式字符串

- `logging` 模块定义的格式字符串字段

字段/属性名称	使用格式	描述
asctime	%(asctime)s	日志事件发生的时间–人类可读时间，如：2003-07-08 16:49:45,896
created	%(created)f	日志事件发生的时间–时间戳，就是当时调用time.time()函数返回的值
relativeCreated	%(relativeCreated)d	日志事件发生的时间相对于logging模块加载时间的相对毫秒数（目前还不知道干嘛用的）
msecs	%(msecs)d	日志事件发生事件的毫秒部分
levelname	%(levelname)s	该日志记录的文字形式的日志级别（'DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL'）
levelno	%(levelno)s	该日志记录的数字形式的日志级别（10, 20, 30, 40, 50）
name	%(name)s	所使用的日志器名称，默认是'root'，因为默认使用的是 rootLogger
message	%(message)s	日志记录的文本内容，通过 msg % args计算得到的
pathname	%(pathname)s	调用日志记录函数的源码文件的全路径
filename	%(filename)s	pathname的文件名部分，包含文件后缀
module	%(module)s	filename的名称部分，不包含后缀
lineno	%(lineno)d	调用日志记录函数的源代码所在的行号
funcName	%(funcName)s	调用日志记录函数的函数名
process	%(process)d	进程ID
processName	%(processName)s	进程名称，Python 3.1新增
thread	%(thread)d	线程ID
threadName	%(thread)s	线程名称

8.3.4 经过配置的日志输出

- 先简单配置下日志器的日志级别

```
logging.basicConfig(level=logging.DEBUG)

logging.debug("This is a debug log.")
logging.info("This is a info log.")
logging.warning("This is a warning log.")
logging.error("This is a error log.")
```

```
logging.critical("This is a critical log.")


"""输出"""
DEBUG:root:This is a debug log.
INFO:root:This is a info log.
WARNING:root:This is a warning log.
ERROR:root:This is a error log.
CRITICAL:root:This is a critical log.
```

所有等级的日志信息都被输出了，说明配置生效了。

- 在配置日志器日志级别的基础上，在配置下日志输出目标文件和日志格式

```
LOG_FORMAT = "%(asctime)s - %(levelname)s - %(message)s"
logging.basicConfig(filename='my.log', level=logging.DEBUG, format=LOG_FORMAT)

logging.debug("This is a debug log.")
logging.info("This is a info log.")
logging.warning("This is a warning log.")
logging.error("This is a error log.")
logging.critical("This is a critical log.")

"""输出"""
2022-01-05 10:37:49,141 - DEBUG - This is a debug log.
2022-01-05 10:37:49,142 - INFO - This is a info log.
2022-01-05 10:37:49,142 - WARNING - This is a warning log.
2022-01-05 10:37:49,142 - ERROR - This is a error log.
2022-01-05 10:37:49,142 - CRITICAL - This is a critical log.
```

此时会发现控制台中已经没有输出日志内容了，但是在python代码文件的相同目录下会生成一个名为’test.log’的日志文件，该文件中的内容为：

```
2022-01-05 10:37:49,141 - DEBUG - This is a debug log.
2022-01-05 10:37:49,142 - INFO - This is a info log.
2022-01-05 10:37:49,142 - WARNING - This is a warning log.
2022-01-05 10:37:49,142 - ERROR - This is a error log.
2022-01-05 10:37:49,142 - CRITICAL - This is a critical log.
```

- 在上面的基础上，我们再来设置下日期/时间格式

```
LOG_FORMAT = "%(asctime)s - %(levelname)s - %(message)s"
DATE_FORMAT = "%m/%d/%Y %H:%M:%S %p"

logging.basicConfig(filename='my.log', level=logging.DEBUG, format=LOG_FORMAT, datefmt=DATE_FORMAT)

logging.debug("This is a debug log.")
logging.info("This is a info log.")
logging.warning("This is a warning log.")
logging.error("This is a error log.")
logging.critical("This is a critical log.")
```

此时会test.log日志文件中看到如下输出内容：

```
05/08/2017 14:29:04 PM - DEBUG - This is a debug log.
05/08/2017 14:29:04 PM - INFO - This is a info log.
05/08/2017 14:29:04 PM - WARNING - This is a warning log.
05/08/2017 14:29:04 PM - ERROR - This is a error log.
05/08/2017 14:29:04 PM - CRITICAL - This is a critical log.
```

掌握了上面的内容之后，已经能够满足我们平时开发中需要的日志记录功能。

8.3.5 自定义 Logger

- 上面的基本使用可以让我们快速上手 logging 模块，但一般并不能满足实际使用，我们还需要自定义 Logger。

  一个系统只有一个 Logger 对象，并且该对象不能被直接实例化，没错，这里用到了单例模式，获取 Logger 对象的方法为 getLogger。
- 注意：这里的单例模式并不是说只有一个 Logger 对象，而是指整个系统只有一个根 Logger 对象，Logger 对象在执行 info()、error() 等方法时实际上调用都是根 Logger 对象对应的 info()、error() 等方法。
- 我们可以创造多个 Logger 对象，但是真正输出日志的是根 Logger 对象。每个 Logger 对象都可以设置一个名字，如果设置logger = logging.getLogger(**name**)，**name** 是 Python 中的一个特殊内置变量，他代表当前模块的名称（默认为 **main**）。则 Logger 对象的 name 为建议使用使用以点号作为分隔符的命名空间等级制度。
- Logger 对象可以设置多个 Handler 对象和 Filter 对象，Handler 对象又可以设置 Formatter 对象。

8.3.6 其他说明

- 几个要说明的内容：
  1. `logging.basicConfig()` 函数是一个一次性的简单配置工具使，也就是说**只有在第一次调用该函数时会起作用**，后续再次调用该函数时完全不会产生任何操作的，多次调用的设置并不是累加操作。
  2. 日志器（Logger）是有层级关系的，上面调用的logging模块级别的函数所使用的日志器是 `RootLogger` 类的实例，其名称为'root'，它是处于日志器层级关系最顶层的日志器，且该实例是以单例模式存在的。

3. 如果要记录的日志中包含变量数据，可使用一个格式字符串作为这个事件的描述消息（`logging.debug`、`logging.info`等函数的第一个参数），然后将变量数据作为第二个参数*args的值进行传递，如: `logging.warning('%s is %d years old.', 'Tom', 10)`，输出内容为`WARNING:root:Tom is 10 years old.`
4. `logging.debug()`，`logging.info()`等方法的定义中，除了`msg`和`args`参数外，还有一个**kwargs参数。它们支持3个关键字参数: `exc_info, stack_info, extra`，下面对这几个关键字参数作个说明。

- 关于`exc_info`，`stack_info`，`extra`关键词参数的说明:
  1. ***exc_info:*** 其值为布尔值，如果该参数的值设置为`True`，则会将异常异常信息添加到日志消息中。如果没有异常信息则添加`None`到日志信息中。
  2. ***stack_info:*** 其值也为布尔值，默认值为`False`。如果该参数的值设置为`True`，栈信息将会被添加到日志信息中。
  3. ***extra:*** 这是一个字典（`dict`）参数，它可以用来自定义消息格式中所包含的字段，但是它的`key`不能与`logging`模块定义的字段冲突。
- 一个例子

  在日志消息中添加exc_info和stack_info信息，并添加两个自定义的字端 ip和user

```
LOG_FORMAT = "%(asctime)s - %(levelname)s - %(user)s[%(ip)s] - %(message)s"
DATE_FORMAT = "%m/%d/%Y %H:%M:%S %p"

logging.basicConfig(format=LOG_FORMAT
                    , datefmt=DATE_FORMAT)
logging.warning("Some one delete the log file."
                , exc_info=True
                , stack_info=True
                , extra={'user': 'Tom', 'ip': '47.98.53.222'}
                )
```

  输出结果:

```
01/05/2022 10:46:59 AM - WARNING - Tom[47.98.53.222] - Some one delete the log file.
NoneType: None
Stack (most recent call last):
  File "/Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/1、Package/13、logging.py", line 94, in <module>
    logging.warning("Some one delete the log file."
```

8.4 logging日志模块相关类及其常用方法介绍

- `logging`四大组件相关的类: `Logger`，`Handler`，`Filter`，`Formatter`。

8.5 封装

- 通用日志类

```
import logging

class Log():
    """Log:   封装日志
    Examples
    ---------------------------------------------------------------------------------------------------
    >>> from options import Log
    >>> logger=Log(logging.DEBUG)
    >>> logger=logger.info(log_file_name='日志')
    >>> logger.info('print test')

    """
    def __init__(self,log_level=logging.INFO):
        '''添加一个日志器'''
        self.log_level=log_level
        self.logger=logging.getLogger()
        self.logger.setLevel(self.log_level)
        self.fmt = self.format()

    def format(self):
        '''设置格式器'''
        self.fmt=logging.Formatter(fmt='文件:[%(filename)s]  级别:[%(levelname)s]  时间:[%(asctime)s] ------->>>>信息:[%(message)s ]')
        return self.fmt

    def hand(self):
        '''添加一个处理器'''
        self.h=logging.StreamHandler()
        self.h.setLevel(logging.WARNING)
        self.h.setFormatter(self.fmt)#将格式器添加到处理器
        self.logger.addHandler(self.h) #将处理器添加到格式器

    def add_file(self,file):
        '''设置文件输出控制台'''

 self.f=logging.FileHandler(filename=file,mode='a',encoding='UTF-8')
        self.f.setLevel(logging.DEBUG)
        self.f.setFormatter(self.fmt)
        self.logger.addHandler(self.f)#将文件输出控制台添加到日志器


```

```
40        def info(self,log_file_name):
41            self.hand()
42            self.add_file(log_file_name)
43            return self.logge
```

9 九、 tqdm 模块

9.1 tqdm简介

9.1.1 地址

- **官网地址**

  > https://pypi.org/project/tqdm/

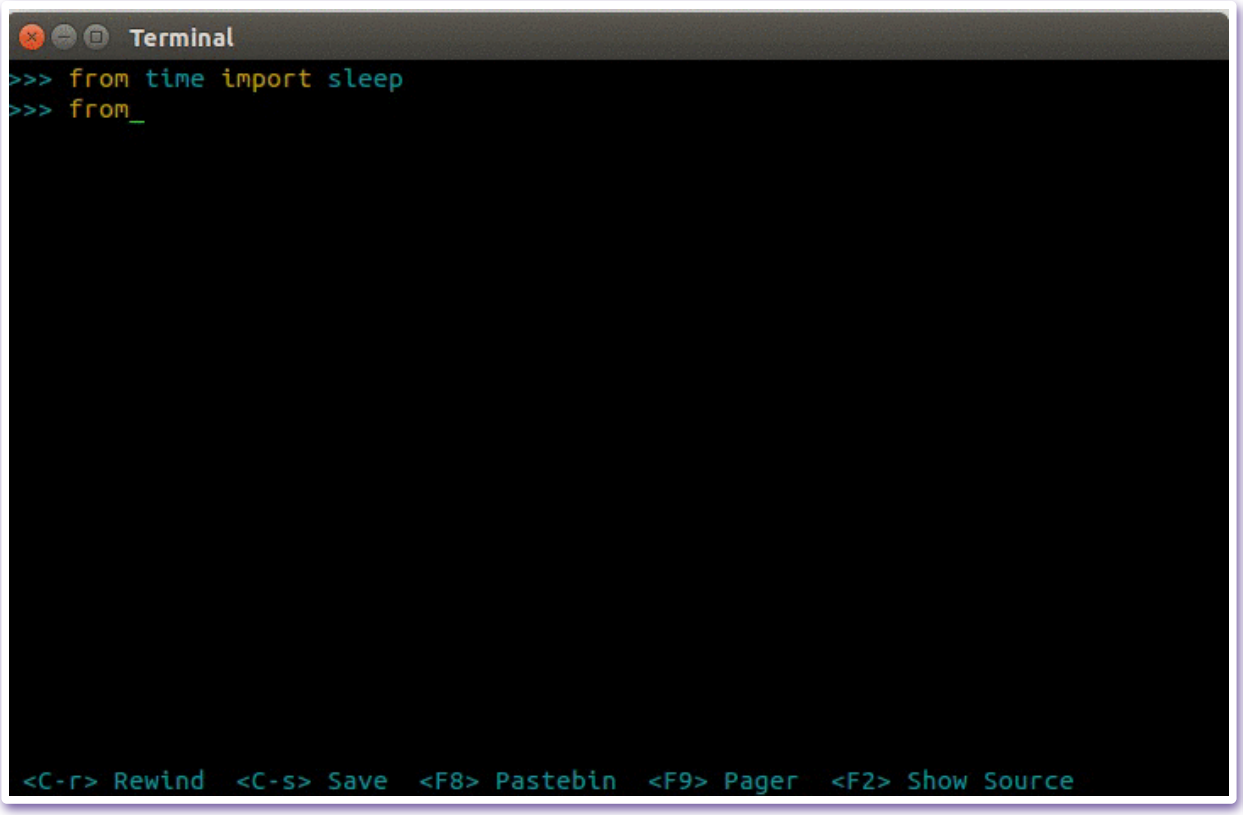
- **github地址**

  > https://github.com/tqdm/tqdm

9.1.2 简介

- Tqdm 是一个快速，可扩展的 `Python` 进度条，可以在 `Python` 长循环中添加一个进度提示信息，用户只需要封装任意的迭代器 `tqdm(iterator)` 。

  总之，它是用来显示进度条的，很漂亮，使用很直观（在循环体里边加个tqdm），而且基本不影响原程序效率。



9.2 tqdm 使用

9.2.1 简单使用

- 简单的demo1：

```
from tqdm import tqdm
import time

d = {
    'loss': 0.2,
    'learn': 0.8
     }

for i in tqdm(range(50)
                , desc='进行中' # desc设置名称
                , ncols=100 # ncols设置进度条长度
                , postfix=d # postfix以字典形式传入详细信息
                ):
    time.sleep(0.1)
    pass
```

运行结果：

```
进行中: 100%|██████████████████████████████████████| 50/50 [00:05<00:00, 9.68it/s, learn=0.8, loss=0.2]
```

- 简单的demo2：

```
import time
from tqdm import tqdm

for i in tqdm(range(100)):
    time.sleep(0.01)
```

运行结果：

```
100%|└└└
```

- 对list的使用

```
from tqdm import tqdm
import time

alist = list('letters')
bar = tqdm(alist)
for letter in bar:
    bar.set_description(f"Now get:  {letter}")
    time.sleep(0.3)
```

运行结果：

```
Now get:  s: 100%|██████████| 7/7 [00:02<00:00,  3.30it/s]
```

- 对list的使用（手动拆解了）

```
from tqdm import tqdm
import time
bar = tqdm(['p1','p2','p3','p4','p5'])
for b in bar:
    time.sleep(0.5)
    bar.set_description("处理{0}中".format(b))
```

运行结果：

```
处理p1中:  20%|██          | 1/5 [00:00<00:02,  2.00it/s]
处理p2中:  40%|████        | 2/5 [00:01<00:01,  1.99it/s]
处理p3中:  60%|██████      | 3/5 [00:01<00:01,  1.99it/s]
处理p4中:  80%|████████    | 4/5 [00:02<00:00,  1.99it/s]
处理p5中: 100%|██████████| 5/5 [00:02<00:00,  1.99it/s]
```

- 传入任意一个列表

```
from tqdm import tqdm
import time

pbar = tqdm(["a", "b", "c", "d"])
for char in pbar:
    pbar.set_description("Processing:【   %s   】" % char)
    time.sleep(0.5)
```

运行结果：

```
Processing:【   d   】: 100%|██████████| 4/4 [00:02<00:00,
 1.99it/s]
```

- 手动设置更新次数，每次更新数量

```
# 一共更新200个，每次更新25，共更新八次with tqdm(total=200) as pbar:  # 一共更新200个    for i in range(8):  # 共更新八次        time.sleep(0.5)        pbar.update(25)  # 每次更新25# 也可以这样pbar = tqdm(total=200)for i in range(8):    time.sleep(0.5)pbar.update(25)pbar.close()
```

运行结果:

```
100%|██████████| 200/200 [00:04<00:00, 49.73it/s]100%|██████████| 200/200 [00:04<00:00, 49.71it/s]
```

9.2.3 结合pandas使用

```
import pandas as pd
import numpy as np
from tqdm import tqdm

df = pd.DataFrame(np.random.randint(0, 100, (10000000, 6)))
df
```

	0	1	2	3	4	5
0	68	57	77	67	72	45
1	91	42	45	10	86	7
2	20	15	95	28	4	36
3	62	90	12	97	9	52
4	36	19	75	45	23	61
...	...	...	...	...	...	...
9999995	23	69	12	73	50	62
9999996	18	2	93	37	60	53
9999997	76	33	17	31	33	1
9999998	46	68	37	87	5	45
9999999	42	99	78	92	67	93

10000000 rows × 6 columns

```
tqdm.pandas(desc="my bar! ")
df.progress_apply(lambda x: x**2)
```

	0	1	2	3	4	5
0	4624	3249	5929	4489	5184	2025
1	8281	1764	2025	100	7396	49
2	400	225	9025	784	16	1296
3	3844	8100	144	9409	81	2704
4	1296	361	5625	2025	529	3721
...	...	...	...	...	...	...
9999995	529	4761	144	5329	2500	3844
9999996	324	4	8649	1369	3600	2809
9999997	5776	1089	289	961	1089	1
9999998	2116	4624	1369	7569	25	2025
9999999	1764	9801	6084	8464	4489	8649

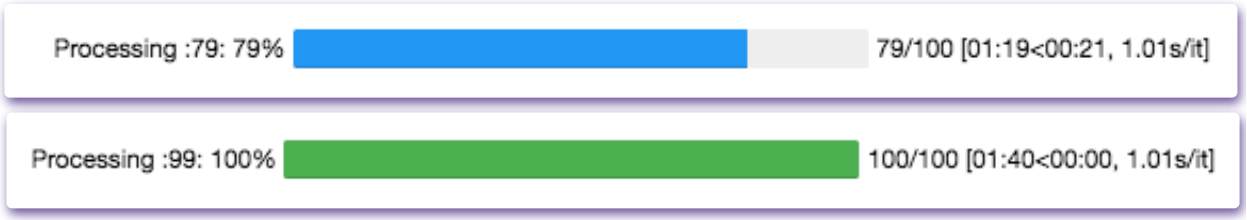
10000000 rows × 6 columns

9.2.4 结合jupyter 使用

- `tqdm` 针对 `jupyter notebook` 添加了专门的进度条美化方法 `tqdm.notebook.tqdm()`，下面是一个简单的例子：

```
from tqdm import notebook
import time

bar = notebook.tqdm(range(100))
for i in bar:
    bar.set_description(f'Processing :{i}')
    time.sleep(0.01)
```

运行结果：



9.2.5 tqdm.write()

- 在使用 `tqdm` 显示进度条的时候，如果代码中存在 `print` 可能会导致输出多行进度条，此时可以将 `print` 语句改为 `tqdm.write`，代码如下：

```python
from tqdm import tqdm

"""主要在命令行使用"""
for i in tqdm(range(10),ascii=True):
    tqdm.write("come on")
    time.sleep(0.1)
```

运行结果：

```

come on

come on

come on

come on

come on

come on

come on

come on

come on

come on

100%|#################################################################| 10/10 [00:01<00:00,  9.68it/s]
>>>

```

10 十、sqlite3

10.1 简介

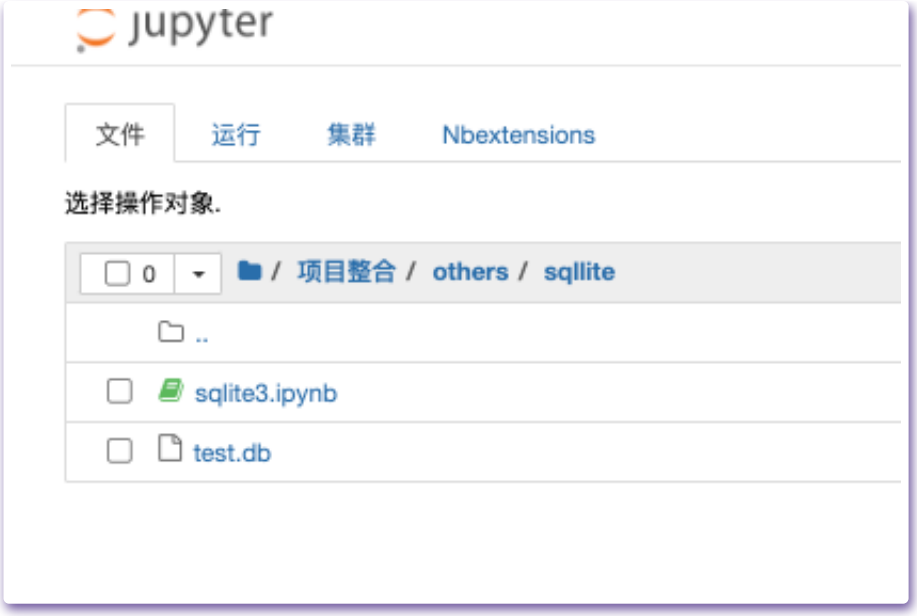
- SQLite，是一款轻型的数据库，它包含在一个相对小的C库中，很多嵌入式产品中使用了它，其中python就嵌入了它。 至2021年已经接近有21个年头，SQLite也迎来了一个版本SQLite 3已经发布。
- 可视化工具：DB Broswer for Sqlite

> http://www.sqlitebrowser.org/

10.2 创建数据库

- 当创建与 SQLite 的连接时，如果它不存在，它将自动创建一个数据库文件。这个数据库文件是在磁盘上创建的；
- 我们还可以使用 :memory: 和 connect 函数在 RAM 中创建数据库。该数据库称为内存数据库。
- 磁盘上创建数据库

```python
import sqlite3

from sqlite3 import Error

def sql_connection():

    try:

        con = sqlite3.connect('test.db')

        print("Connection is established: Database is created in memory")

    except Error:

        print(Error)

    finally:

        con.close()

sql_connection()
```



- 内存上创建数据库

```
import sqlite3

from sqlite3 import Error

def sql_connection():

    try:

        con = sqlite3.connect('test.db')

        print("Connection is established: Database is created in memory")

    except Error:

        print(Error)

    finally:

        con.close()

sql_connection()
```

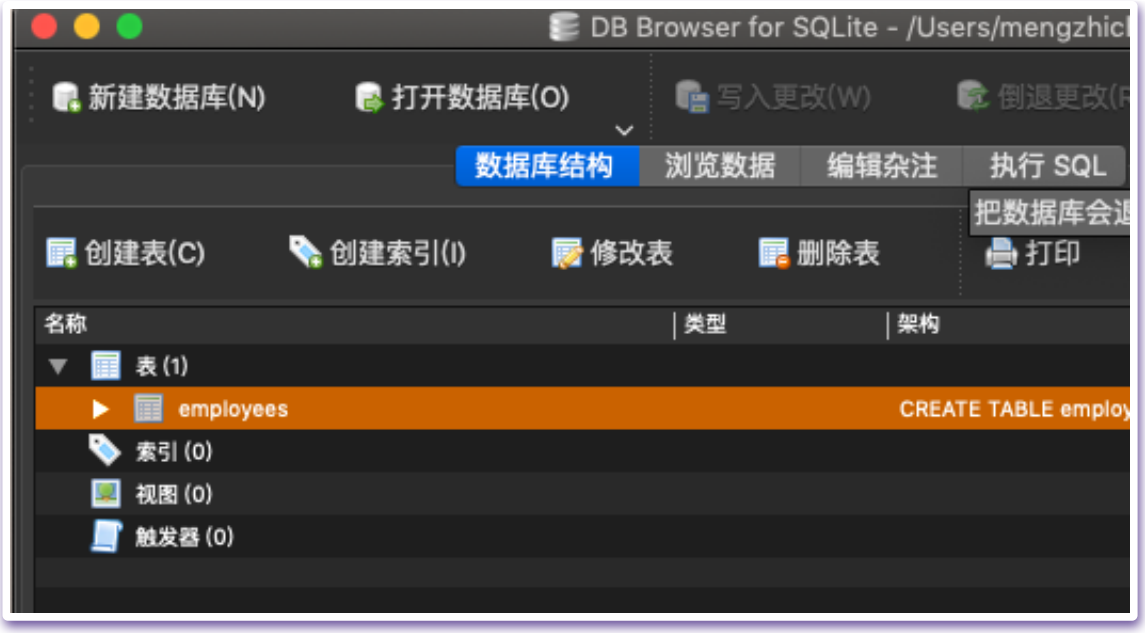
10.3 创建表

- 要在 SQLite3 中创建表，可以在execute()方法中使用 Create Table 查询。考虑以下步骤：
    1. 创建连接对象。
    2. 从连接对象创建一个游标对象。
    3. 使用游标对象，以创建表查询为参数调用execute方法。

让我们创建具有以下属性的员工：

> employees (id, name, salary, department, position, hireDate)

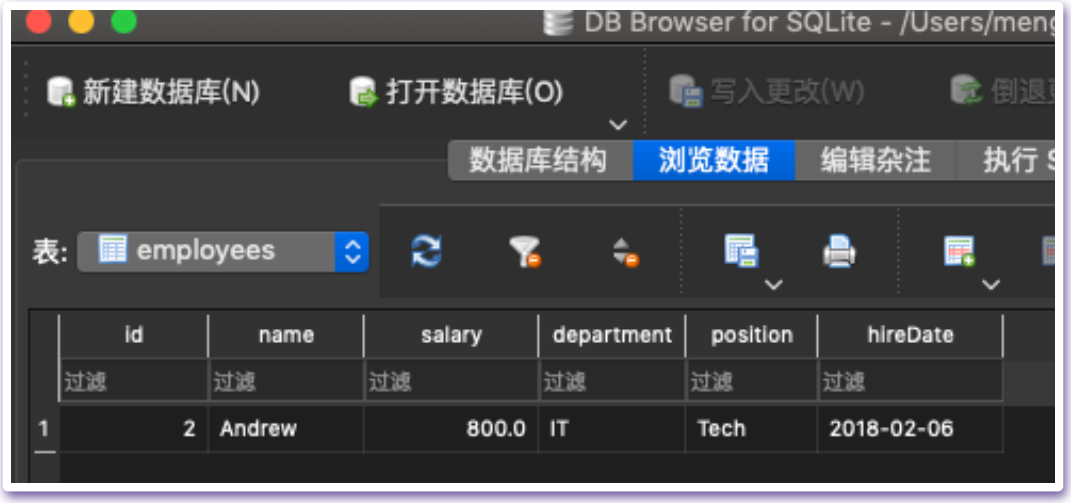
```
import sqlite3

from sqlite3 import Error

def sql_connection():

    try:

        con = sqlite3.connect('test.db')

        return con

    except Error:

        print(Error)

def sql_table(con):

    cursorObj = con.cursor()

    cursorObj.execute("""
        CREATE TABLE employees(
                            id integer PRIMARY KEY,
                            name text,
                            salary real,
                            department text,
                            position text,
                            hireDate text)

    """)

    con.commit()

con = sql_connection()

sql_table(con)
```



10.4 插入数据

- 向表中插入数据

```
import sqlite3

con = sqlite3.connect('test.db')

def sql_insert(con, entities):

    cursorObj = con.cursor()
    con.commit()

entities = (2, 'Andrew', 800, 'IT', 'Tech', '2018-02-06')
sql_insert(con, entities)
```



10.5 更新表

- 要更新表，只需创建一个连接，然后使用该连接创建一个游标对象，最后在execute()方法中使用 UPDATE 语句。

  假设我们要更新 id 等于 2 的员工的姓名。为了更新，我们将使用 UPDATE 语句和 id 等于 2 的员工。我们将使用 WHERE 子句作为选择该员工的条件。

```
import sqlite3

con = sqlite3.connect('test.db')

def sql_update(con):

    cursorObj = con.cursor()

    cursorObj.execute('UPDATE employees SET name = "Rogers" where id = 2')

    con.commit()

sql_update(con)
```

10.6 获取所有数据

- featchall()

```
import sqlite3

con = sqlite3.connect('test.db')

def sql_fetch(con):

    cursorObj = con.cursor()
```

```python

    cursorObj.execute('SELECT * FROM employees')

    rows = cursorObj.fetchall()

    for row in rows:

        print(row)

sql_fetch(con)

"""输出"""
(2, 'Rogers', 800.0, 'IT', 'Tech', '2018-02-06')
```

10.7 列出所有表

- 要列出 SQLite3 数据库中的所有表，应该查询 sqlite_master 表，然后使用 *fetchall()* 从 SELECT 语句中获取结果。

  sqlite_master 是 SQLite3 中的主表，存放所有的表。

```python
import sqlite3

con = sqlite3.connect('test.db')

def sql_fetch(con):

    cursorObj = con.cursor()

    cursorObj.execute('SELECT name from sqlite_master where type="table"')

    print(cursorObj.fetchall())

sql_fetch(con)


"""输出"""
[('employees',)]
```

10.8 创建-删除表

- 创建时检查表是否存在

> create table if not exists table_name (column1, column2, …, columnN)

- 删除时检查表是否存在

> drop table if exists table_name

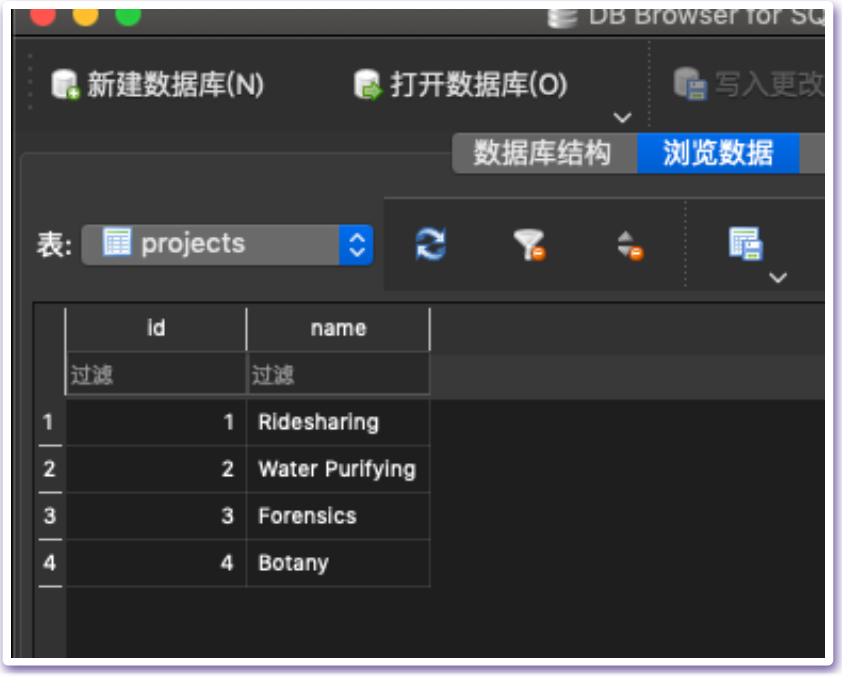
- 检查我们要访问的表是否存在

> cursor.('SELECT name from sqlite_master WHERE type = "table" AND name = "employees"')
>
> print(cursor.fetchall())

10.9 批量插入

- executemany插入多行

```
import sqlite3

con = sqlite3.connect('test.db')

cursor = con.cursor()

cursor.execute('create table if not exists projects(id integer, name text)')

data = [(1, "Ridesharing"), (2, "Water Purifying"), (3, "Forensics"), (4, "Botany")]

cursor.executemany("INSERT INTO projects VALUES(?, ?)", data)

con.commit()
```



11 十一、pytesseract

- 安装pytesseract-ocr

  mac:

```
//先安装依赖库libpng, jpeg, libtiff, leptonica
brew install leptonica

//安装tesseract的同时安装训练工具
brew install --with-training-tools tesseract

//安装tesseract的同时安装所有语言，语言包比较大，如果安装的话时间较长，建议不安装，按需选择
brew install  --all-languages tesseract

//安装tesseract，并安装训练工具和语言
brew install --all-languages --with-training-tools tesseract

//只安装tesseract，不安装训练工具
brew install  tesseract
```

  win:

> 安装包地址： https://github.com/UB-Mannheim/tesseract/wiki

  配置环境变量

  安装依赖包：

> pip install pytesseract
>
> pip install pillow

- 下载语言库

  `chi_sim.traineddata`:中文简体

  `eng.traineddata`:英文

> https://github.com/tesseract-ocr/tessdata

- 使用

```
from PIL import Image
import pytesseract
text = pytesseract.image_to_string(Image.open('./eng.png')
                                   ,lang='eng' # 指定语言，chi_sim为简体中文
                                  )
print(text)
```

12 十二、schedule

- 安装：

> pip install schedule

12.1 按时间间隔执行定时任务

- 主要注意例如day，days这种单复数的区别

  且schedule方法是串行的，也就是说，如果各个任务之间时间不冲突，那是没问题的；如果时间有冲突的话，会串行的执行命令。

```
import schedule
from datetime import datetime

def job(name):
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"Now_time:{now_time}, her name is : {name}")


name = "张三"
schedule.clear() # 清空任务
# schedule.every(3).seconds.do(job, name)            # 创建一个每三秒执行一次的任务
# schedule.every().minutes.do(job,name)              # 每隔1分钟执行一次
# schedule.every(10).minutes.do(job,name)              # 每隔10分钟执行一次
# schedule.every().hour.do(job, name)                  # 每隔一小时执行一次
# schedule.every().day.do(job,name)                    # 每隔一天执行一次
# schedule.every(3).days.do(job,name)                  # 每隔三天执行一次
# schedule.every().day.at("10:30").do(job, name)       # 每天上午10:30
schedule.every().day.at('17:12').do(job, name)       # 每天下午17:12
# schedule.every(5).to(10).days.do(job, name)          # 每隔5到10天执行一次
# schedule.every().monday.do(job, name)                # 每周一的这个时候执行一次任务
# schedule.every().wednesday.at("13:15").do(job, name) #每周三13:15执行一次任务


while True:
    schedule.run_pending() # 运行所有可以运行的任务
    time.sleep(1)
```

```

"""输出"""
Now_time:2023-07-10 17:12:00, her name is : 张三
Now_time:2023-07-11 17:12:00, her name is : 张三


```

12.2 装饰器：通过 @repeat() 装饰静态方法

- 示例代码

```
from datetime import datetime
import schedule
from schedule import every, repeat, run_pending

schedule.clear()

@repeat(every(3).seconds)
def task1():
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"NowTime : {now_time}, 任务1, 每3秒执行一次")

@repeat(every(10).seconds)
def task2():
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"NowTime : {now_time}, 任务2, 每10秒执行一次")


while True:
    run_pending()


"""输出"""
NowTime : 2023-07-10 17:18:41, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:18:44, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:18:47, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:18:48, 任务2, 每10秒执行一次      1
NowTime : 2023-07-10 17:18:50, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:18:53, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:18:56, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:18:58, 任务2, 每10秒执行一次       1
NowTime : 2023-07-10 17:18:59, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:19:02, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:19:05, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:19:08, 任务2, 每10秒执行一次        1
```

12.3 传递参数

- 不使用装饰器传参

```
from datetime import datetime
import schedule

schedule.clear()

def task1(string):
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"NowTime : {now_time}, {string}")

def task2(string):
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"NowTime : {now_time}, 任务2, 每10秒执行一次")

schedule.every(3).seconds.do(task1,string='任务1, 每3秒执行一次')
schedule.every(10).seconds.do(task1,string='任务2, 每10秒执行一次')

while True:
    run_pending()


"""输出"""
NowTime : 2023-07-10 17:24:25, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:28, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:31, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:32, 任务2, 每10秒执行一次
NowTime : 2023-07-10 17:24:34, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:37, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:40, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:42, 任务2, 每10秒执行一次
NowTime : 2023-07-10 17:24:43, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:46, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:49, 任务1, 每3秒执行一次
NowTime : 2023-07-10 17:24:52, 任务2, 每10秒执行一次
```

- 使用装饰器传参

```
from datetime import datetime
import schedule
from schedule import every, repeat, run_pending

schedule.clear()

@repeat(every(3).seconds,'任务1, 每3秒执行一次')
def task(string):
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"NowTime : {now_time},{string} ")
```

```python

@repeat(every(10).seconds,'任务2，每10秒执行一次')
def task2(string):
    now_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f"NowTime : {now_time}, {string}")

while True:
    run_pending()


"""输出"""
NowTime : 2023-07-10 17:27:53,任务1，每3秒执行一次
NowTime : 2023-07-10 17:27:56,任务1，每3秒执行一次
NowTime : 2023-07-10 17:27:59,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:00，任务2，每10秒执行一次
NowTime : 2023-07-10 17:28:02,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:05,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:08,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:10，任务2，每10秒执行一次
NowTime : 2023-07-10 17:28:11,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:14,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:17,任务1，每3秒执行一次
NowTime : 2023-07-10 17:28:20，任务2，每10秒执行一次
```

12.4 取消定时任务

- schedule.cancel_job(jobname)

```python
import schedule
import sys


i = 0
def some_task():
    global i
    i += 1
    print(i)
    if i == 5:
        schedule.cancel_job(job)
        print('cancel job')
        sys.exit(0)


job = schedule.every().second.do(some_task)
while True:
    schedule.run_pending()


"""输出"""
```

```
1
2
3
4
5
cancel job
An exception has occurred, use %tb to see the full traceback.
```

12.5 在指定时间执行一次任务

- 如，在最近的一分钟里30秒的时候执行一次任务

```python
import time
import schedule

def task():
    print('这个任务只执行一次')


def job_that_executes_once():
    task()
    return schedule.CancelJob

schedule.every().minute.at(':30').do(job_that_executes_once)
while True:
    schedule.run_pending()
    time.sleep(1)

"""输出"""
这个任务只执行一次
```

12.6 根据标签检索、取消任务

- 检索

```python
# 检索所有任务：schedule.get_jobs()
import schedule

def greet(name):
    print('Hello {}'.format(name))


schedule.every().day.do(greet, 'Andrea').tag('daily-tasks', 'friend')
schedule.every().hour.do(greet, 'John').tag('hourly-tasks', 'friend')
```

```python
schedule.every().hour.do(greet, 'Monica').tag('hourly-tasks', 'customer')
schedule.every().day.do(greet, 'Derek').tag('daily-tasks', 'guest')
friends = schedule.get_jobs('friend')
print(friends)


"""输出"""
[Every 1 day do greet('Andrea') (last run: [never], next run: 2023-07-11 17:39:47)
 Every 1 hour do greet('John') (last run: [never], next run: 2023-07-10 18:39:47)]
```

- 取消

```python
# 取消所有任务：schedule.clear()
import schedule

def greet(name):
    print('Hello {}'.format(name))
    if name == 'Cancel':
        schedule.clear('second-tasks')
        print('cancel second-tasks')


schedule.every().second.do(greet, 'Andrea').tag('second-tasks', 'friend')
schedule.every().second.do(greet, 'John').tag('second-tasks', 'friend')
schedule.every().hour.do(greet, 'Monica').tag('hourly-tasks', 'customer')
schedule.every(5).seconds.do(greet, 'Cancel').tag('daily-tasks', 'guest')
while True:
    schedule.run_pending()


"""输出"""
Hello Andrea
Hello John
Hello Andrea
Hello John
Hello Andrea
Hello John
Hello Andrea
Hello John
Hello Cancel
cancel second-tasks
```

12.7 运行任务到某时间

- until

```python
import schedule
from datetime import datetime, timedelta, time


def job():
    print('working...')


schedule.every().second.until('23:59').do(job)  # 今天23:59停止
schedule.every().second.until('2030-01-01 18:30').do(job)  # 2030-01-01 18:30停止
schedule.every().second.until(timedelta(hours=8)).do(job)  # 8小时后停止
schedule.every().second.until(time(23, 59, 59)).do(job)  # 今天23:59:59停止
schedule.every().second.until(datetime(2030, 1, 1, 18, 30, 0)).do(job)  # 2030-01-01 18:30停止
while True:
    schedule.run_pending()
```

13 十三、rich

- 提供了丰富的样式化和美化功能

```python
from rich.console import Console
from rich.theme import Theme
from rich import traceback

traceback.install()


shrek_dict = {
    "movie_title": "Shrek",
    "release_year": 2001,
    "characters": {
        "main_characters": {
            "Shrek": {
                "description": "An ogre with a heart of gold.",
                "voice_actor": "Mike Myers",
                "quotes": [
                    "Get out of my swamp!",
                    "Ogres are like onions.",
                    "Better out than in, I always say."
                ]
            },
            "Fiona": {
                "description": "A princess with a secret.",
                "voice_actor": "Cameron Diaz",
                "quotes": [
                    "I'm not a princess, I'm an ogress!",
                    "This is one of those onion things, isn't it?"
                ]
            },
            "Donkey": {
                "description": "A talkative and loyal companion.",
                "voice_actor": "Eddie Murphy",
                "quotes": [
                    "What did I say? I'm just a love machine.",
                    "Shrek! What are you doing in my closet?"
                ]
            }
        },
        "supporting_characters": {
            "Lord_Farquaad": {
                "description": "A short-tempered and power-hungry lord.",
                "voice_actor": "John Lithgow",
                "quotes": [
                    "People of DuLoc, I am your lord now!",
```

```
                        "Mirror, mirror, on the wall, is this not the most perfect kingdom of them all?"
                    ]
                },
                "Dragon": {
                    "description": "A fire-breathing dragon with a soft spot for Donkey.",
                    "voice_actor": "Frank Welker",
                    "quotes": [
                        "RAWR!",
                        "I'm a mother, I have needs!"
                    ]
                }
            }
        },
        "scenes": {
            "opening": "Shrek wakes up in his swamp and scares away intruders.",
            "fairy_tale_characters": "Shrek meets various fairy tale characters in his swamp.",
            "dragon_tower": "Fiona is trapped in a tower guarded by the Dragon.",
            "final_battle": "Shrek and Donkey rescue Fiona and confront Lord Farquaad."
        },
        "themes": [
            "Acceptance of oneself",
            "True beauty is on the inside",
            "Love conquers all"
        ],
        "sequels": [
            "Shrek 2",
            "Shrek the Third",
            "Shrek Forever After"
        ],
        "trivia": {
            "fun_facts": [
                "Shrek was the first animated film to win an Academy Award for Best Animated Feature.",
                "The voice of Shrek was originally going to be provided by Chris Farley, but he passed away before recording was completed."
            ],
            "box_office": "The film grossed over $484 million worldwide."
        }
    }


print(shrek_dict)
from rich import print # 覆盖原有的print
print(shrek_dict)
```

```python


custom_theme = Theme( {
    "info": "bold cyan",
    "warning": "bold yellow",
    "error": "bold red",
})

console = Console(theme=custom_theme)

def compare_numbers(a,b):
    console.log(log_locals=True)
    if a>b:
        console.print(f'{a} is greater than {b}', style="bold green")
    elif a<b:
        console.print(f'{a} is less than {b}', style="bold red")
    else:
        console.print(f'{a} is equal to {b}', style="bold yellow")


compare_numbers(5,10)
compare_numbers(10,5)
compare_numbers(5,5)

compare_numbers(5,"mzc")

```

14 pathlib

14.1 常用属性和函数

- 常用方法以及描述：

方法	描述	功能模块
`Path.home()`	返回用户主目录路径	**创建路径**
`Path.cwd()`	返回当前工作目录	**创建路径**
`Path.resolve()`	返回路径的绝对路径	**创建路径**
`Path.joinpath(*other)`	拼接路径	**创建路径**
`Path.with_name(name)`	替换路径的文件名部分	**创建路径**
`Path.with_suffix(suffix)`	替换路径的扩展名部分	**创建路径**
`Path.exists()`	检查路径是否存在	**检查路径**
`Path.is_file()`	检查路径是否是文件	**检查路径**
`Path.is_dir()`	检查路径是否是目录	**检查路径**
`Path.mkdir(parents=False, exist_ok=False)`	创建目录，支持递归	**文件操作**
`Path.rmdir()`	删除空目录	**文件操作**
`Path.unlink()`	删除文件	**文件操作**
`Path.rename(target)`	重命名文件或目录	**文件操作**
`Path.replace(target)`	替换文件或目录	**文件操作**
`Path.stat()`	获取文件或目录的属性	**文件操作**
`Path.chmod(mode)`	修改文件或目录的权限	**文件操作**
`Path.iterdir()`	遍历目录下的文件和子目录	**遍历目录**
`Path.glob(pattern)`	根据模式匹配文件	**遍历目录**
`Path.rglob(pattern)`	递归匹配目录下的文件	**遍历目录**
`Path.name`	返回路径的文件名部分	**路径信息**
`Path.stem`	返回文件名（不带扩展名）	**路径信息**

`Path.suffix`	返回文件的扩展名	**路径信息**
`Path.suffixes`	返回文件的所有扩展名	**路径信息**
`Path.parent`	返回路径的父目录	**路径信息**
`Path.parents`	返回路径的所有父目录	**路径信息**
`Path.read_text()`	读取文件内容为字符串	**读写文件**
`Path.write_text(data)`	将字符串写入文件	**读写文件**
`Path.read_bytes()`	读取文件内容为字节	**读写文件**
`Path.write_bytes(data)`	将字节写入文件	**读写文件*

14.2 示例

- 识别当前目录用法

```
from pathlib import Path

# 识别当前工作目录     下的 data/test.log
file_path = Path(__file__).parent / "data" / "test.log"

```

- 其他常用代码

```
from pathlib import Path

# # 一、创建路径
# 创建当前目录的路径对象
current_path = Path('.')
current_path
# 创建绝对路径对象
home_path = Path.home()  # 用户主目录(家目录)
home_path
# 创建相对路径对象
file_path = Path('data', 'test.log')
file_path


# # 二 拼接路径
# 使用 / 运算符拼接路径
new_path = Path('.') / 'data' / 'test.log'
new_path

# #  三、获取路径信息
```

```python
# 获取文件名
new_path.name
# 获取文件的扩展名
new_path.suffix
# 获取父目录
new_path.parent
# 获取绝对路径
new_path.resolve()

# # 四、检查路径
# 检查路径是否存在
new_path.exists()
# 检查是否是文件
new_path.is_file()
# 检查是否是目录
new_path.is_dir()


# # 五、文件系统操作
# 创建目录
new_path.mkdir(parents=True, exist_ok=True)
# 删除文件
new_path.unlink()  # 如果路径是文件，删除文件
# 遍历目录
for file in Path('.').iterdir():
    print(file)


# # 六、读写文件
# 写入文件
with new_path.open('w') as file:
    file.write("Hello, Pathlib!")
# 读取文件
with new_path.open('r') as file:
    content = file.read()
    print(content)
```

15 十五、 pydantic

- 示例代码(用户管理系统)

```python
from pydantic import BaseModel, field_validator, model_validator, ValidationError, Field, EmailStr
from rich import traceback

traceback.install()

# 定义地址模型
class Address(BaseModel):
    city: str
    zip_code: str

    # 自定义验证器：检查邮政编码格式
    @field_validator("zip_code")
    def check_zip_code(cls, value):
        if not value.isdigit() or len(value) != 5: # 若不是数字或长度不等于5
            raise ValueError("Zip code must be 5 digits")
        return value

# 定义用户模型
class User(BaseModel):
    id: int
    name: str
    email: EmailStr  # 验证电子邮件格式
    age: int
    address: Address  # 嵌套模型

    # 字段验证器：检查年龄是否为正数
    @field_validator("age")
    def check_age(cls, value):
        if value < 0:
            raise ValueError("Age cannot be negative")
        return value

    # 模型验证器：检查用户是否成年
    @model_validator(mode="after")
    def check_adult(self) -> "User":
        if self.age < 18:
            raise ValueError("User must be an adult (age >= 18)")
        return self

    # 配置选项：禁止额外字段
    model_config = {
        "extra": "forbid"
    }

```

```python
# 示例数据
user_data = {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "age": 25,
    "address": {
        "city": "Wonderland",
        "zip_code": "12345"
    }
}

# 创建用户对象
try:
    user = User(**user_data)
    print("用户对象创建成功: ")
    print(user)
except ValidationError as e:
    print("验证失败: ", e)

# 序列化为字典
user_dict = user.model_dump()
print("\n序列化为字典: ")
print(user_dict)

# 序列化为 JSON
user_json = user.model_dump_json()
print("\n序列化为 JSON: ")
print(user_json)

# 测试额外字段
invalid_data = {
    "id": 2,
    "name": "Bob",
    "email": "bob@example.com",
    "age": 30,
    "address": {
        "city": "Neverland",
        "zip_code": "67890"
    },
    "extra_field": "value"  # 额外字段，会触发验证错误
}

try:
    user = User(**invalid_data)
except ValidationError as e:
    print("\n额外字段验证失败: ", e)

# 测试自定义验证器
invalid_age_data = {
    "id": 3,
    "name": "Charlie",
```

```
    "email": "charlie@example.com",
    "age": -5,  # 负数年龄，会触发验证错误
    "address": {
        "city": "Dreamland",
        "zip_code": "54321"
    }
}

try:
    user = User(**invalid_age_data)
except ValidationError as e:
    print("\n年龄验证失败：", e)

# 测试嵌套模型的验证器
invalid_zip_code_data = {
    "id": 4,
    "name": "David",
    "email": "david@example.com",
    "age": 20,
    "address": {
        "city": "Fairyland",
        "zip_code": "abcde"  # 无效的邮政编码，会触发验证错误
    }
}

try:
    user = User(**invalid_zip_code_data)
except ValidationError as e:
    print("\n邮政编码验证失败：", e)

# 测试模型验证器
invalid_adult_data = {
    "id": 5,
    "name": "Eve",
    "email": "eve@example.com",
    "age": 16,  # 未成年，会触发验证错误
    "address": {
        "city": "Gotham",
        "zip_code": "98765"
    }
}

try:
    user = User(**invalid_adult_data)
except ValidationError as e:
    print("\n成年验证失败：", e)
```